Machine Learning - 2

A Project Report

Submitted to

greatlearning
Learning for Life

By

Chaitra M

in Partial Fulfillment of
PGPDS.O.Apr25.A

TEXAS McCombs
The University of Texas at Austin
McCombs School of Business

TABLE OF CONTENT

Context **7**
Objective **8**
Data Description **8**
Data Overview **10**
Exploratory Data Analysis **14**
Univariate Analysis 14
Observation on continent 14
Observation on Job Experience 15
Observation on Job Requirement 15
Observation on Employment Region 16
Fig 5: Observation on Employment Region 16
Observation on Wage Unit 16
Observation on Case ID 17
Observation on Number Of Employees 17
Bivariant Analysis 18
Observation on Correlation Matrix on Numerical Variables 18
Observation on Education of Employee and Case Status 19
Correlation between Education of Employee and Region of Employment 20
Observation on Education of Employee and Region of Employment 21
Observation on Continent and Case Status 22
Observation on Job Experience and case Status 23
Observation on Job Requirement and Job Training 24
Observation on Prevailing Wage and Case Status 25
Observation on Prevailing_Wage and Region of Employement 26
Observation on Unit Of Wages and Case Status 27
Observation on Job Training and Case Study 28
Observation on Position Type and Case Status 29
Data Processing **30**
Missing value treatment 30
Feature Engineer 30
Preparing Data For Modeling 32
Bagging - Model Building and Hyperparameter Tuning **33**
Decision Tree Model 33
Decision Tree Classifier with Hyperparameter Tuning 34
Bagging Classifier 37
Bagging Classifier with Hyperparameter Tuning 38
Random Forest Classifier 40
Random Forest Classifier with Hyperparameter Tuning 41
Boosting - Model Building and Hyperparameter Tuning **44**
AdaBoost Classifier 44

AdaBoost Classifier with Hyperparameter Tuning 45
Gradient Boosting Classifier 48
Gradient Boost Classifier with Hyperparameter Tuning 49
XGBoost Classifier 52
XG Boost Classifier with Hyperparameter Tuning 53
Stacking Classifier 56
Model Performance Comparison and Conclusions On Bagging and Boosting 57
Base models comparison 57
Tuned models comparison 58
Model Performance Comparison 58
Feature importance of: Tuned Gradient Boosting Classifier 60
Visualization of Decision Tree 61
Feature Importance of: Decision Tree 61
Model Building For Oversampling and Undersampling 62
Model Building - Oversampled Data 63
Model Building - Undersampled Data 64
Hyperparameter Tuning 65
Tuning AdaBoostClassifier model with Undersampled data 65
Tuning AdaBoostClassifier model with Oversampled data 66
Tuning Gradient Boosting model with Undersampled Data 67
Tuning Gradient Boosting model with Oversampled data 68
Model Comparison and Final Model Selection 69
Feature Importance of OverSampled AdaBoost Classifier 70
Actionable Insights and Business Recommendations 71

LIST OF FIGURES

Fig 1: Observation on Continent
Fig 2: Observation on Education Level
Fig 3: Observation on Job Experience
Fig 4: Observation On Job Requirement
Fig 5: Observation on Employment Region
Fig 6: Observation on Wage Unit
Fig 7: Observation on Case ID
Fig 8: Observation on Number of Employees
Fig 9: Observation of Correlation Matrix on Numerical Variable
Fig 10: Observation on Education of Employee and Case Status
Fig 11: Correlation between Education of Employee and Region of Employment
Fig 12: Observation on Education of Employee and Region of Employment
Fig 13: Observation on Continent and Case Status
Fig 14: Observation on Job Experience and case Status
Fig 15: Observation on Job Requirement and Job Training
Fig 16: Observation on Prevailing Wage and Case Status
Fig 17: Observation on Prevailing Wage and Case Status
Fig 18: Observation on Prevailing Wage and Region of Employment
Fig 19: Observation on Unit Of Wages and Case Status
Fig 20: Observation on Job Training and Case Study
Fig 21: Observation on Position Type and Case Status
Fig 22: Checking Missing value
Fig 23: Outlier Detection
Fig 24: Treating Outliers
Fig 25: Confusion Matrix of Decision Tree Classifier's Performance on Test Data
Fig 26: Confusion Matrix of Tuned Decision Tree Classifier's Performance on Test Data
Fig 27: Feature Importance of Tuned Decision True Model
Fig 28: Confusion Matrix of Bagging Classifier's Performance on Test Data
Fig 29: Confusion Matrix of Tuned Bagging Classifier's Performance on Test Data
Fig 30: Confusion Matrix of Random Forest Classifier's Performance on Test Data
Fig 31: Confusion Matrix of Tuned RandomForest Classifier's Performance on Test Data
Fig 32: Feature Importance of Tuned Random Forest Classifier
Fig 33: Confusion Matrix of AdaBoost Classifier's Performance on Test Data
Fig 34: Confusion Matrix of Tuned AdaBoost Classifier's Performance on Test Data
Fig 35: Feature Importance of Tuned AdaBoost
Fig 36: Confusion Matrix of Gradient Boosting Classifier's Performance on Test Data

Fig 37: Confusion Matrix of Tuned Gradient Boost Classifier's Performance on Test Data
Fig 38: Feature Importance of Tuned Gradient Boost Classifier
Fig 39: Confusion Matrix of XGBoosting Classifier's Performance on Test Data
Fig 40: Confusion Matrix of Tuned XG Boost Classifier's Performance on Test Data
Fig 41: Feature Importance of Tuned Gradient Boost Classifier
Fig 42: Confusion Matrix of Stacking Classifier's Performance on Test Data
Fig 43: Base models comparison
Fig 44: Tuned models comparison
Fig 45: Comparing the performance of all the model on training data
Fig 46: Comparing the performance of all the model on testing data
Fig 47: Feature importance of: Tuned Gradient Boosting Classifier
Fig 48: Visualization of Decision Tree
Fig 49: Feature Importance of Decision Tree
Fig 50: Model Comparison of Oversampled and Undersampled data
Fig 51: Feature Importance of OverSampled AdaBoost Classifier

LIST OF TABLES

Table 1: Data Description
Table 2: The Performance of a Training and Test Data Of Decision Tree
Table 3: The Tuned Decision Tree Classifier's Performance of Training and Testing Data
Table 4: The Performance of a Training and Test Data Of Bagging Classifier
Table 5: The Tuned Bagging Classifier's Performance of Training and Testing Data
Table 6: The Performance of a Training and Test Data Of Random Forest Classifier
Table 7: The Tuned Random Forest Performance of a Training and Test Data
Table 8: The Performance of a Training and Test Data Of AdaBoost Classifier
Table 9: The Tuned AdaBoost Classifier Performance of a Training and Test Data
Table 10: The Performance of a Training and Test Data Of Gradient Boost Classifier
Table 11: The Tuned Gradient Boost Classifier Performance of a Training and Test Data
Table 12: The Performance of a Training and Test Data Of XGBoost Classifier
Table 13: The Tuned XG Boost Classifier Performance of a Training and Test Data
Table 14: The Performance of a Training and Test Data Of Stacking Classifier
Table 15: The Tuned AdaBoost Classifier Performance of a Training and Validation Set(Undersample Data)
Table 16: The Tuned AdaBoost Classifier Performance of a Training and Validation Set(OverSample Data)
Table 17: The Tuned Gradient Boost Classifier Performance of a Training and Validation Set(UnderSample Data)
Table 18: The Tuned Gradient Boost Classifier Performance of a Training and Validation Set(OverSample Data)
Table 19: The Tuned AdaBoost Classifier Performance of an oversampled Test Data

Context

Business communities in the United States are facing high demand for human resources, but one of the constant challenges is identifying and attracting the right talent, which is perhaps the most important element in remaining competitive. Companies in the United States look for hard-working, talented, and qualified individuals both locally as well as abroad.

The Immigration and Nationality Act (INA) of the US permits foreign workers to come to the United States to work on either a temporary or permanent basis. The act also protects US workers against adverse impacts on their wages or working conditions by ensuring US employers' compliance with statutory requirements when they hire foreign workers to fill workforce shortages. The immigration programs are administered by the Office of Foreign Labor Certification (OFLC).

OFLC processes job certification applications for employers seeking to bring foreign workers into the United States and grants certifications in those cases where employers can demonstrate that there are not sufficient US workers available to perform the work at wages that meet or exceed the wage paid for the occupation in the area of intended employment.

Objective

In FY 2016, the OFLC processed 775,979 employer applications for 1,699,957 positions for temporary and permanent labor certifications. This was a nine percent increase in the overall number of processed applications from the previous year. The process of reviewing every case is becoming a tedious task as the number of applicants is increasing every year.

The increasing number of applicants every year calls for a Machine Learning based solution that can help in shortlisting the candidates having a higher chance of VISA approval. OFLC has hired the firm EasyVisa for data-driven solutions. You, as a data scientist at EasyVisa, have to analyze the data provided and, with the help of a classification model:

Facilitate the process of visa approvals. Recommend a suitable profile for the applicants for whom the visa should be certified or denied based on the drivers that significantly influence the case status.

Data Description

The data contains the different attributes of the employee and the employer. The detailed data dictionary is given below.

Column Name	Description
case_id	ID of each visa application
continent	Information of continent the employee
education_of_employee	Information of education of the employee
has_job_experience	Does the employee have any job experience? Y= Yes; N = No
requires_job_training	Does the employee require any job training? Y = Yes; N = No
no_of_employees	Number of employees in the employer's company
yr_of_estab	Year in which the employer's company was established
region_of_employment	Information of foreign worker's intended region of employment in the US.

prevailing_wage	Average wage paid to similarly employed workers in a specific occupation in the area of intended employment. The purpose of the prevailing wage is to ensure that the foreign worker is not underpaid compared to other workers offering the same or similar service in the same area of employment.
unit_of_wage	Unit of prevailing wage. Values include Hourly, Weekly, Monthly, and Yearly.
full_time_position	Is the position of work full-time? Y = Full-Time Position; N = Part-Time Position
case_status	Flag indicating if the Visa was certified or denied

Table 1: Data Description

Data Overview

1. Import the required packages
2. Loaded the EasyVisa dataset using **read_csv()**.
3. Copied the dataframe to another variable.

```
df.head()
```

	case_id	continent	education_of_employee	has_job_experience	requires_job_training	no_of_employees	yr_of_estab	region_of_employment	prevailing_wage	unit_of_wage	full_time_position	case_status
0	EZYV01	Asia	High School	N	N	14513	2007	West	592.2029	Hour	Y	Denied
1	EZYV02	Asia	Master's	Y	N	2412	2002	Northeast	83425.6500	Year	Y	Certified
2	EZYV03	Asia	Bachelor's	N	Y	44444	2008	West	122996.8600	Year	Y	Denied
3	EZYV04	Asia	Bachelor's	N	N	98	1897	West	83434.0300	Year	Y	Denied
4	EZYV05	Africa	Master's	Y	N	1082	2005	South	149907.3900	Year	Y	Certified

4. Used the **head()** to get the first five rows of data.

```
df.tail()
```

	case_id	continent	education_of_employee	has_job_experience	requires_job_training	no_of_employees	yr_of_estab	region_of_employment	prevailing_wage	unit_of_wage	full_time_position	case_status
25475	EZYV25476	Asia	Bachelor's	Y	Y	2601	2008	South	77092.57	Year	Y	Certified
25476	EZYV25477	Asia	High School	Y	N	3274	2006	Northeast	279174.79	Year	Y	Certified
25477	EZYV25478	Asia	Master's	Y	N	1121	1910	South	146298.85	Year	N	Certified
25478	EZYV25479	Asia	Master's	Y	Y	1918	1887	West	86154.77	Year	Y	Certified
25479	EZYV25480	Asia	Bachelor's	Y	N	3195	1960	Midwest	70876.91	Year	Y	Certified

5. Used the **tail()** to get the last five rows of data.
6. Checked the shape of the dataset using the **shape method.**
    a. The number of rows is 25480.
    b. The number of columns is 12.
7. Checked the data types of all the columns in the dataset using **info().**
    a. There are 12 columns
        i. 2 of these columns are integer datatypes.
        ii. 9 of them being objects.
        iii. Only 1 of them being a float.
    b. There seems to be no missing values for any of these columns.
    c. We should convert the object type columns to categories, which reduces the memory required to store the dataframe.
    d. We need to drop the case_id column from the dataset.
8. Converted all the object columns to category columns.
9. Dropped the case_id column from the dataset.

10. The values present in the categorical columns are
    a. Now there are 8 categorical column among which
    i. The continent column has a total of 6 unique values ['Africa', 'Asia', 'Europe', 'North America', 'Oceania', 'South America'].
    ii. The education_of_employee column has a total of 4 unique values ['Bachelor's', 'Doctorate', 'High School', 'Master's'].
    iii. The has_job_experience column has a total of 2 unique values ['N', 'Y'].
    iv. The requires_job_training column has a total of 2 unique values ['N', 'Y'].
    v. The region_of_employment column has a total of 5 unique values ['Island', 'Midwest', 'Northeast', 'South', 'West'].
    vi. The full_time_position column has a total of 2 unique values ['N', 'Y'].
    vii. The unit_of_wage column has a total of 4 unique values ['Hour', 'Month', 'Week', 'Year'].
    viii. The case_status column has a total of 2 unique values ['Certified', 'Denied'].
11. Checked the statistical summary of the given data set using **describe().**
    a. The min value for no_of_employees cant be -26
    b. Most employees have a bachelor's degree
    c. Most of the employment is from the Northeast region.
    d. The most popular continent which applicants are from is Asia.
    e. Average prevailing wage is $74456.There's also a very huge difference in 75th percentile and maximum value
    f. There's an extremely large gap between the 75th percentile and the max value of no_of_employees
    g. Most of the employees have job experience.
    h. Most of the employees are certified employees.

```
df.describe(include='all').T
# setting include='all' will get the statistical summary for both the numerical and categorical variables.
```

	count	unique	top	freq	mean	std	min	25%	50%	75%	max
continent	25480	6	Asia	16861	NaN	NaN	NaN	NaN	NaN	NaN	NaN
education_of_employee	25480	4	Bachelor's	10234	NaN	NaN	NaN	NaN	NaN	NaN	NaN
has_job_experience	25480	2	Y	14802	NaN	NaN	NaN	NaN	NaN	NaN	NaN
requires_job_training	25480	2	N	22525	NaN	NaN	NaN	NaN	NaN	NaN	NaN
no_of_employees	25480.0	NaN	NaN	NaN	5667.04321	22877.928848	-26.0	1022.0	2109.0	3504.0	602069.0
yr_of_estab	25480.0	NaN	NaN	NaN	1979.409929	42.366929	1800.0	1976.0	1997.0	2005.0	2016.0
region_of_employment	25480	5	Northeast	7195	NaN	NaN	NaN	NaN	NaN	NaN	NaN
prevailing_wage	25480.0	NaN	NaN	NaN	74455.814592	52815.942327	2.1367	34015.48	70308.21	107735.5125	319210.27
unit_of_wage	25480	4	Year	22962	NaN	NaN	NaN	NaN	NaN	NaN	NaN
full_time_position	25480	2	Y	22773	NaN	NaN	NaN	NaN	NaN	NaN	NaN
case_status	25480	2	Certified	17018	NaN	NaN	NaN	NaN	NaN	NaN	NaN

12. Converted negative values present in no_of_employees column to positive using abs().
13. There are no duplicate values in the data set which is determined using **duplicated().sum()**.
14. There are no null values in the data set which is determined using **isnull().sum()**.
15. case_status is encoded as “certified” to 1 and "denied" as 0.

Exploratory Data Analysis

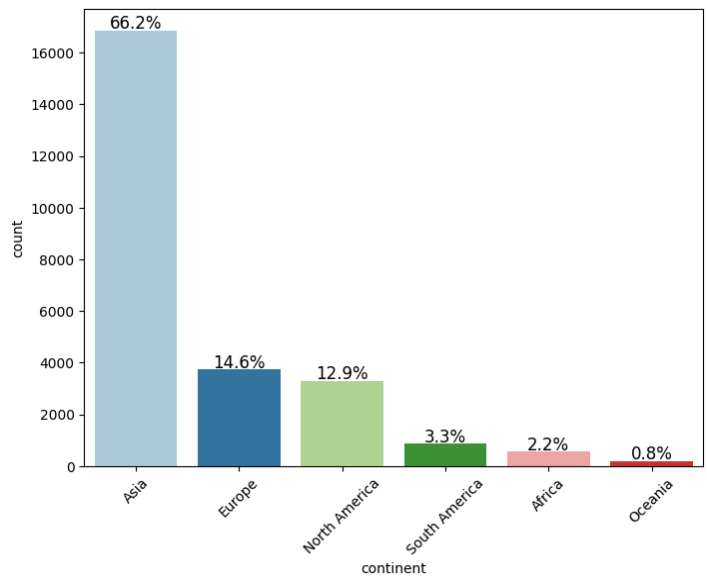
Univariate Analysis

Observation on continent

Observations:

- Europe and North America have the close numbers of applicants 14.6% and 12.9% respectively.
- The most of the visa applicants are from Asia 66%, which makes it the highest population of the continent.
- The smallest number of the visa applicants are from Oceania 0.8%, which makes it the lowest population of the continent.

Fig 1: Observation on Continent

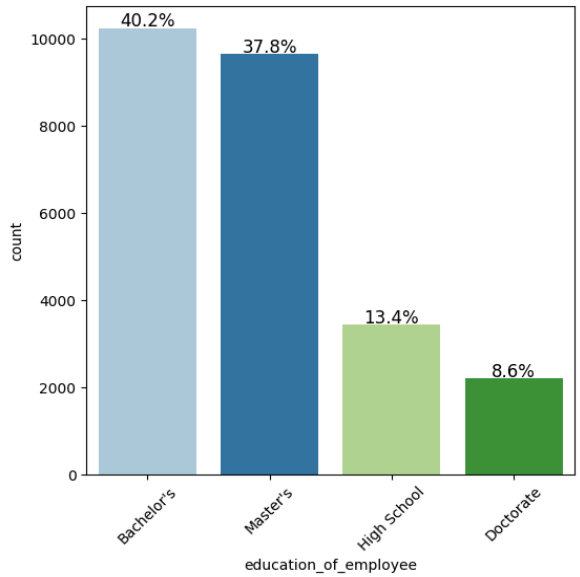


Observation on Education Level

Observations:

- Most of the applicants have either bachelor's degree(40.2%) or Master's degree(37.8%).
- Only 8.6% of the applicants have a doctorate degree.
- 13.4% of the applicants have completed till high school.

Fig 2: Observation on Education Level

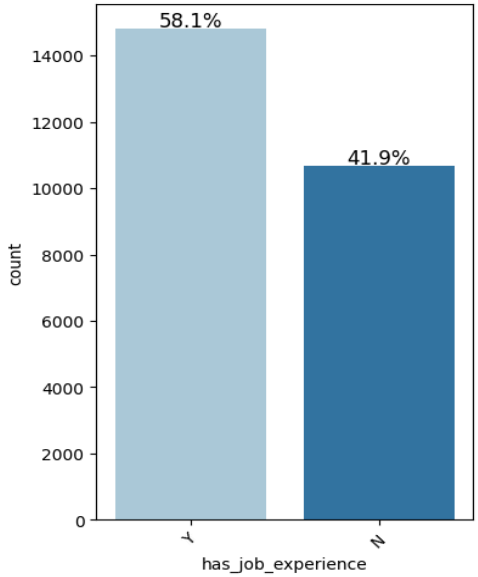


Observation on Job Experience

Fig 3: Observation on Job Experience

Observations:

- Almost 58% of the applicants have no job.

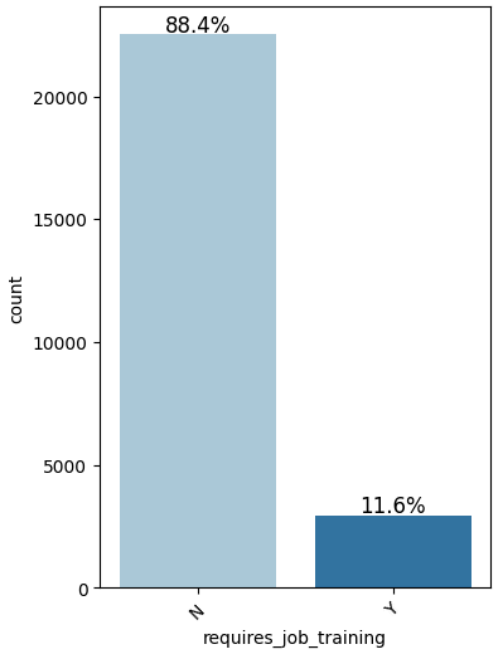


Observation on Job Requirement

Fig 4: Observation On Job Requirement

Observations:

- Most of the jobs don't require any training for the applicants.

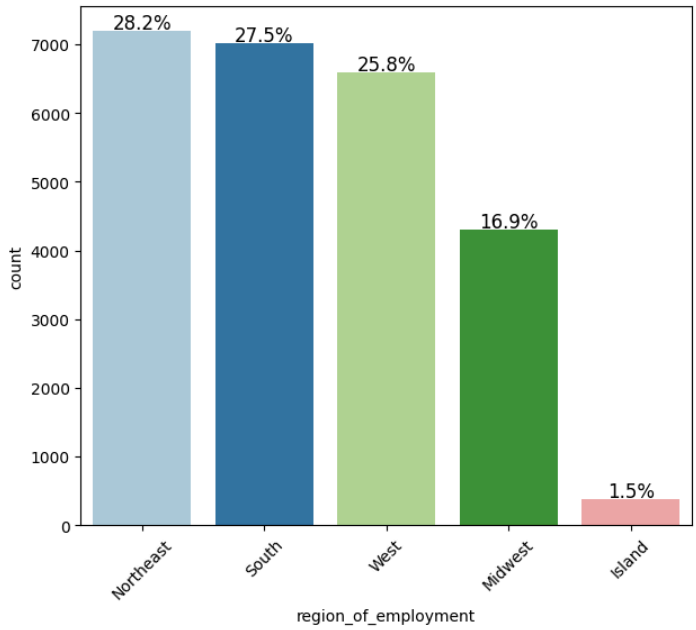


Observation on Employment Region

Fig 5: Observation on Employment Region

Observations:

- The island region has the lowest number of work visa applicants which is 1.5%.
- The majority of the applicants are from Northeast, South and West regions of the United States.

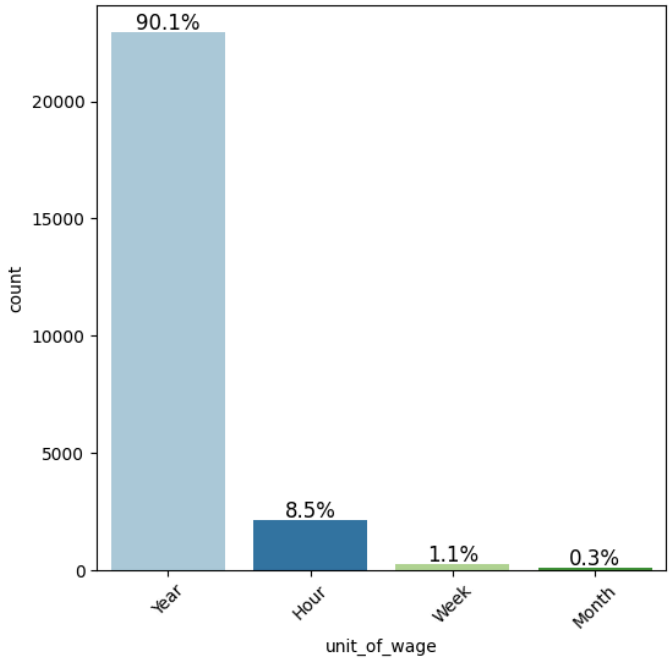


Observation on Wage Unit

Fig 6: Observation on Wage Unit

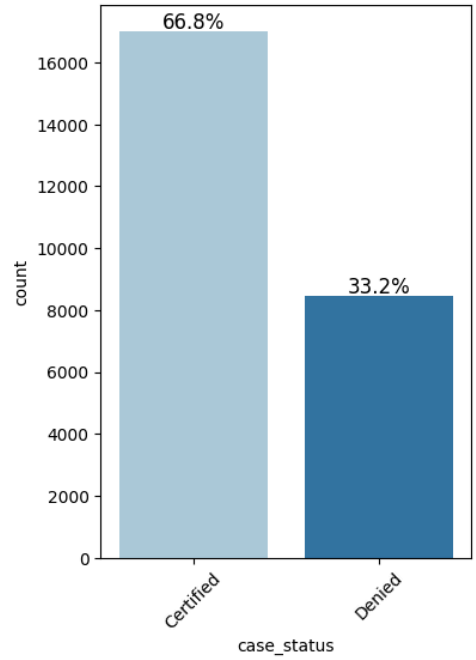
Observations:

- The majority of the employees receive the wages yearly.



Observation on Case ID

Fig 7: Observation on Case ID

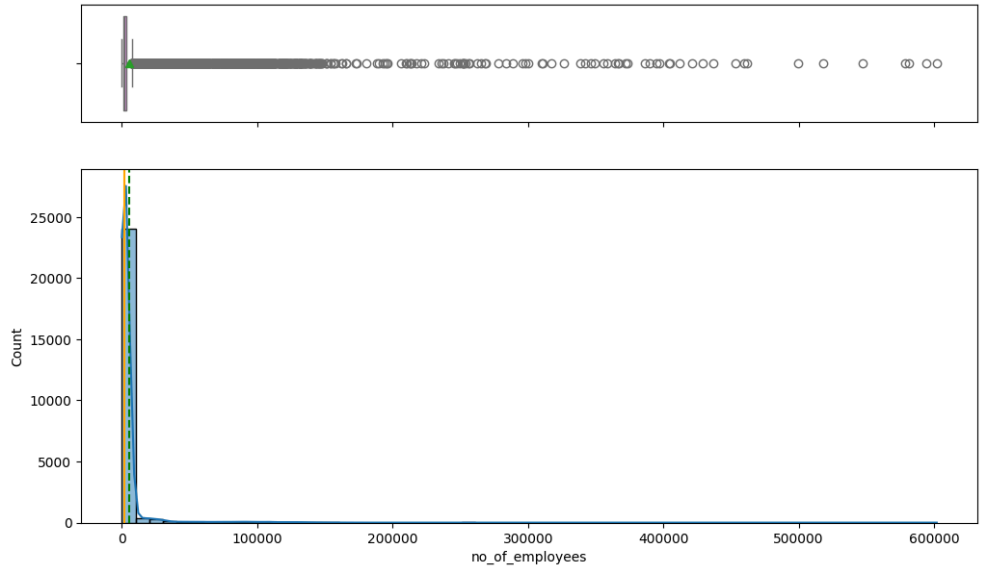


Observations:

- 66.8% of the visa applicants are certified while 33.2% are denied.

Observation on Number Of Employees

Fig 8: Observation on Number of Employees



Observations

- We can see the outliers in the dataset.
- Right skewed distribution.

Bivariant Analysis

Observation on Correlation Matrix on Numerical Variables

	no_of_employees	yr_of_estab	prevailing_wage
no_of_employees	1.000000	-0.017770	-0.009523
yr_of_estab	-0.017770	1.000000	0.012342
prevailing_wage	-0.009523	0.012342	1.000000

Observations:

In the provided correlation matrix, the correlation coefficients are as follows:

- Correlation between "no_of_employees" and "yr_of_estab" = -0.017770
- Correlation between "no_of_employees" and "prevailing_wage" = -0.009523
- Correlation between "yr_of_estab" and "prevailing_wage" = 0.012342
- The correlation coefficient between these variables is very close to zero
- This indicates a weak correlation or almost no linear relationship

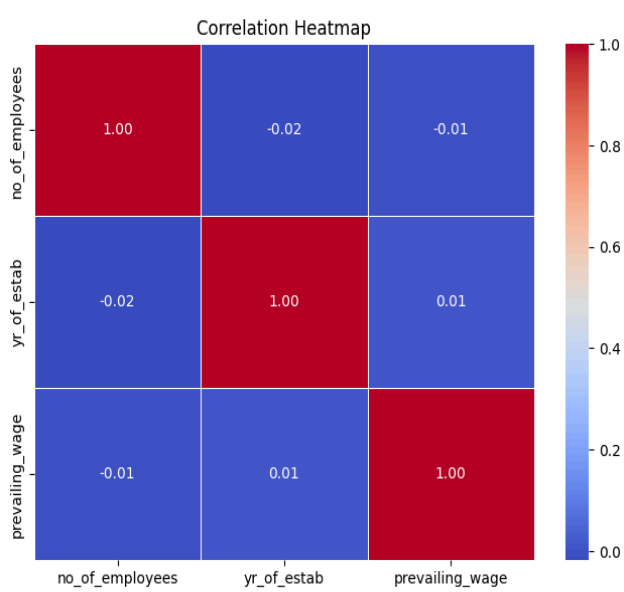


Fig 9: Observation of Correlation Matrix on Numerical Variable

Observation on Education of Employee and Case Status

case_status	Certified	Denied
education_of_employee		
High School	0.340351	0.659649
Bachelor's	0.622142	0.377858
Master's	0.786278	0.213722
Doctorate	0.872263	0.127737

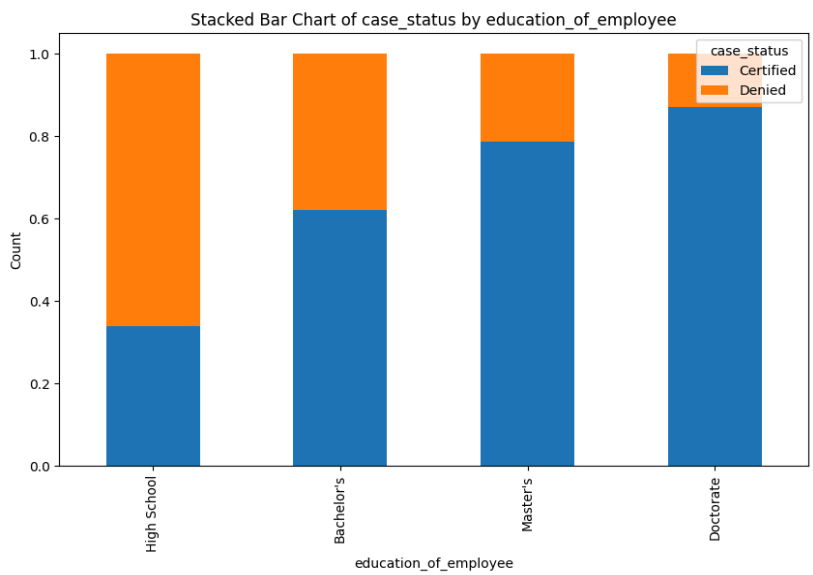


Fig 10: *Observation on Education of Employee and Case Status*

Observations:

- It seems like the higher the degree one has, the more likely your visa will be accepted.
- High school education has less visa certification than those with doctorate education.

Correlation between Education of Employee and Region of Employment

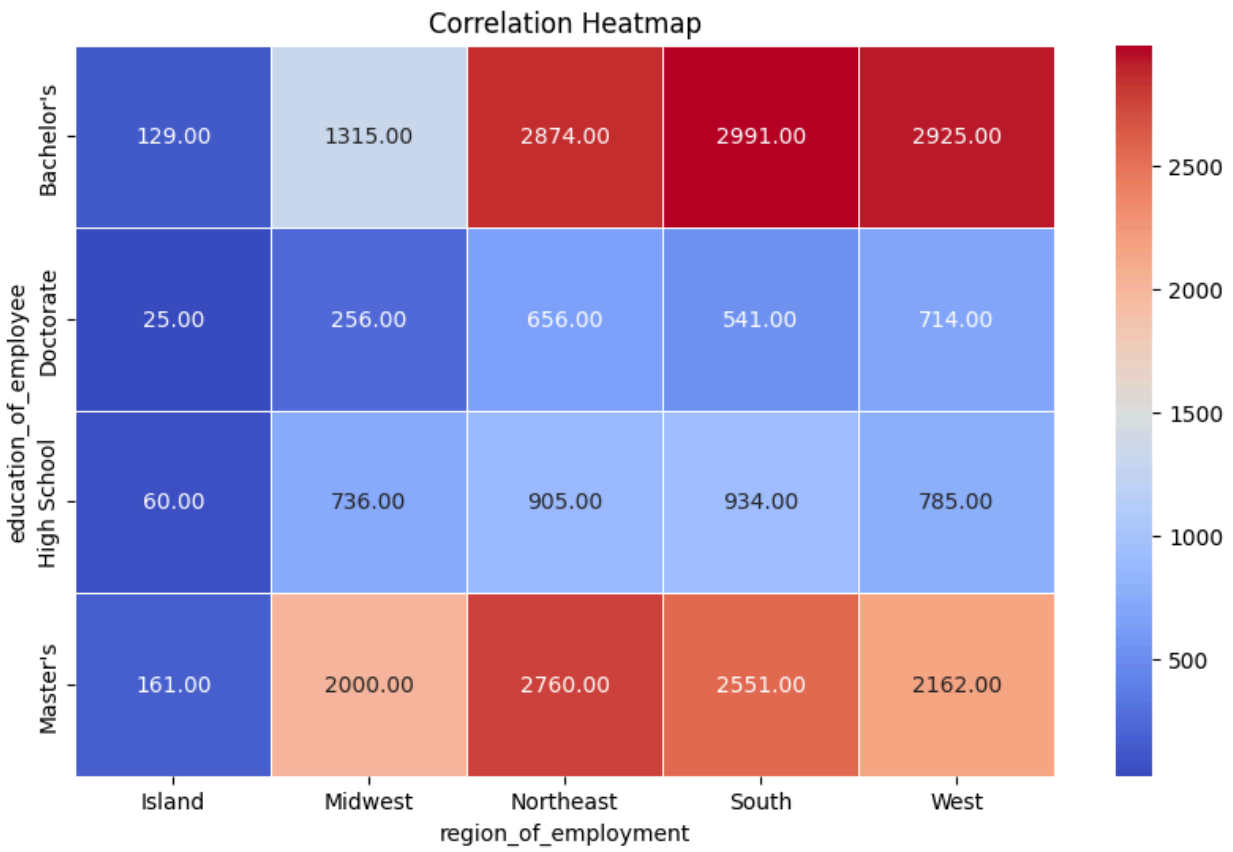


Fig 11: Correlation between Education of Employee and Region of Employment

Observations

- The requirement for the applicants who have passed high school is mostly in the South region, followed by the Northeast region.
- The requirement for Bachelor's is mostly in the South region, followed by the West region.
- The requirement for Master's is most in the Northeast region, followed by the South region.
- The requirement for Doctorate's is mostly in the West region, followed by the Northeast region.

Observation on Education of Employee and Region of Employment

case_status	Certified	Denied
region_of_employment		
Island	0.602667	0.397333
West	0.622533	0.377467
Northeast	0.629048	0.370952
South	0.700157	0.299843
Midwest	0.755282	0.244718

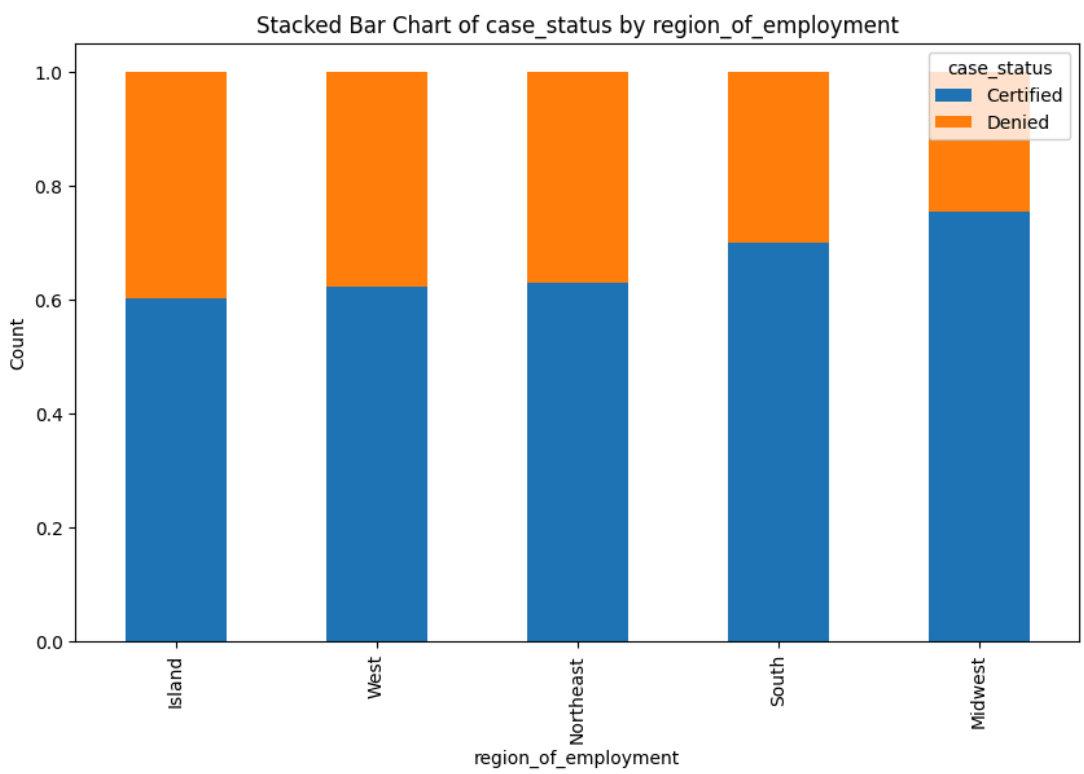


Fig 12: Observation on Education of Employee and Region of Employment

Observations:

- The Midwest sees the highest number of visa certifications but it's not the most picked from any level of education; the South sees the next highest.

Observation on Continent and Case Status

case_status	Certified	Denied
continent		
South America	0.578638	0.421362
North America	0.618773	0.381227
Oceania	0.635417	0.364583
Asia	0.653105	0.346895
Africa	0.720508	0.279492
Europe	0.792337	0.207663

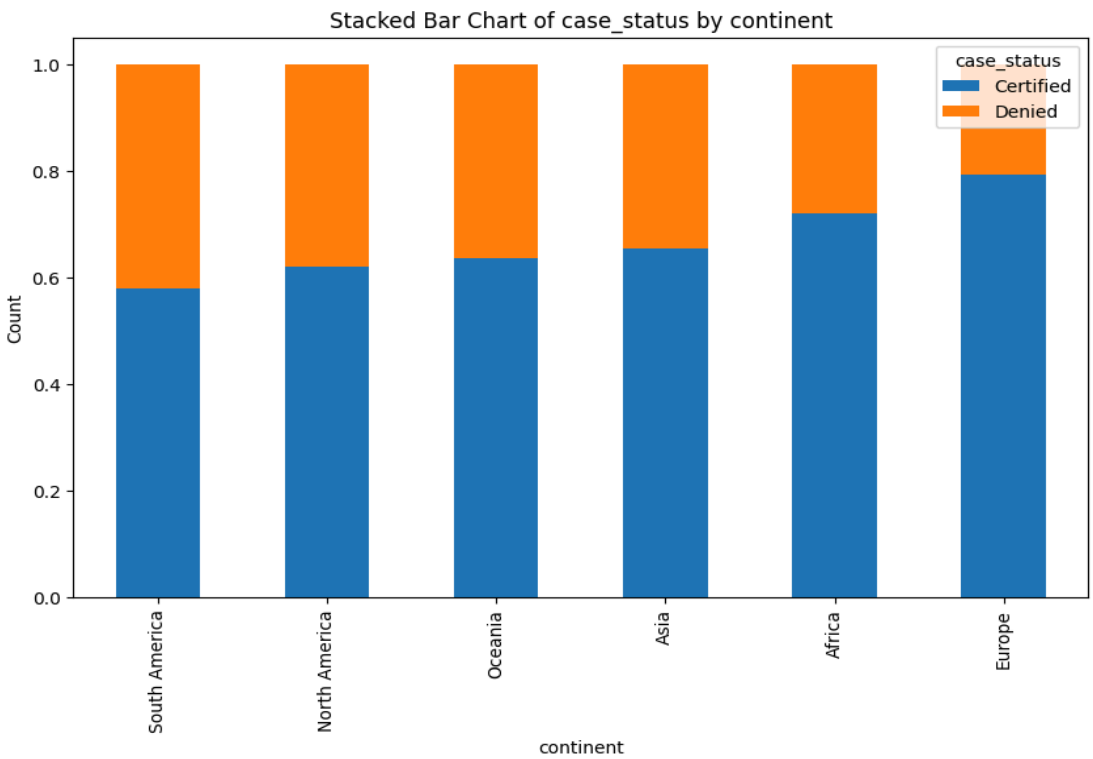


Fig 13: Observation on Continent and Case Status

Observations:

- Europe has the highest visa acceptance followed by Africa.
- Asia has the 3rd highest visa certification but has the highest number of applications.

Observation on Job Experience and case Status

case_status	Certified	Denied
has_job_experience		
N	0.561341	0.438659
Y	0.744764	0.255236

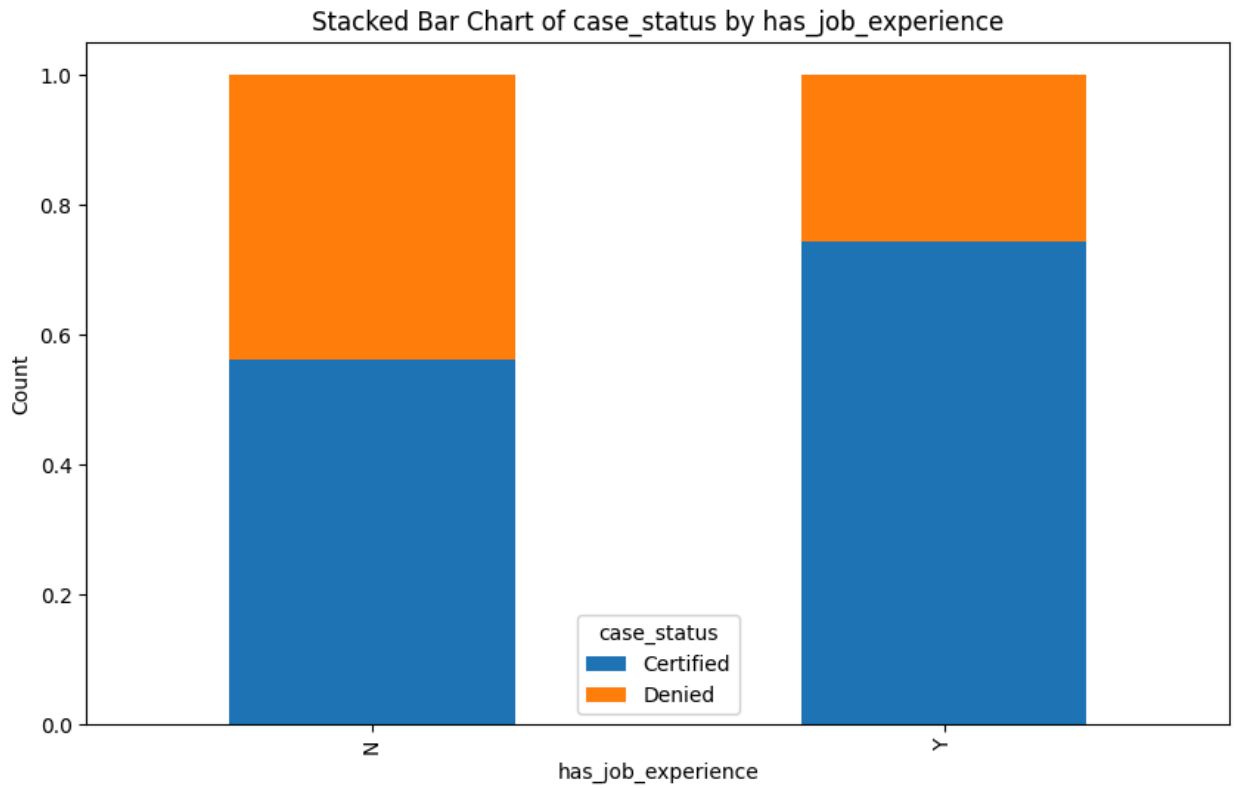


Fig 14: Observation on Job Experience and case Status

Observations

- Those that have job experience have a higher chance of their visa being certified with about 74%.
- About 56% of people who did not have a job experience also got their visa certified.

Observation on Job Requirement and Job Training

requires_job_training	N	Y
has_job_experience		
N	0.841731	0.158269
Y	0.914539	0.085461

Fig 15: Observation on Job Requirement and Job Training

Observations:

- Most candidates, whether they have job experience or not, do not require job training (blue portion is largest).
- A small portion of candidates in both groups (experienced and non-experienced) still require training (orange portion).
- The difference in training requirement between experienced and non-experienced candidates is not very large — in both cases, training needs are relatively low.

Observation on Prevailing Wage and Case Status

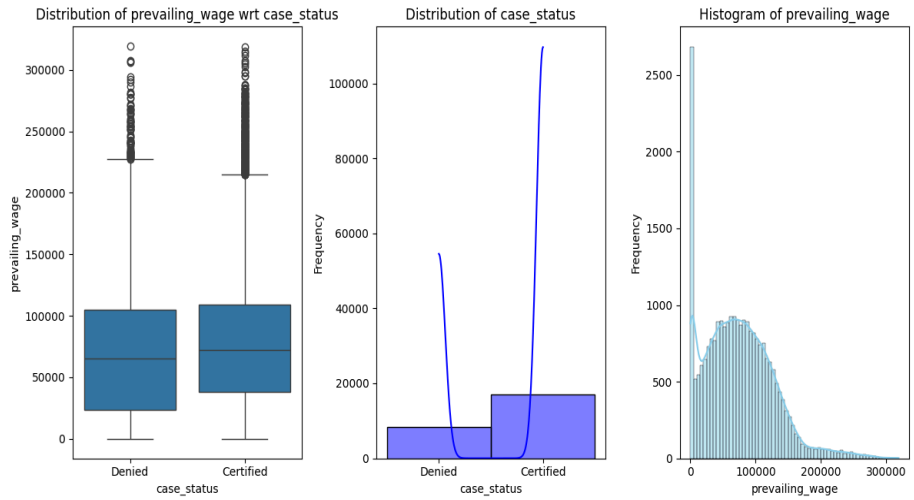


Fig 16: Observation on Prevailing Wage and Case Status

Observations:

- The median wage for the certified applications is slightly higher than the denied application
- The prevailing wage is similar across all regions.
- • Distribution of wages is right skewed, indicating a long tail of jobs with annual salaries above 200,000 dollars.
- • The ratio of approved to denied applications remains equivalent across the range of wages.

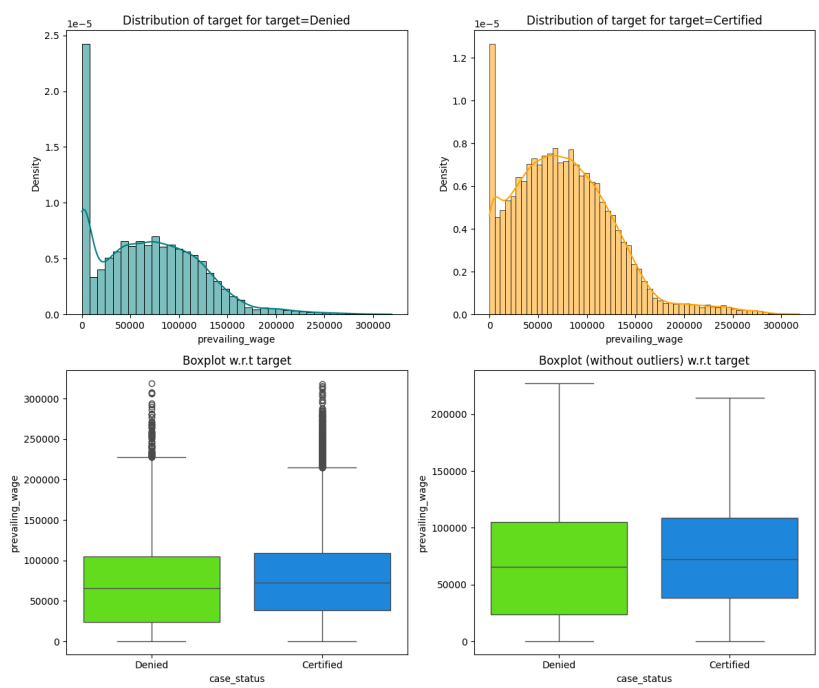


Fig 17: Observation on Prevailing Wage and Case Status

Observation on Prevailing_Wage and Region of Employement

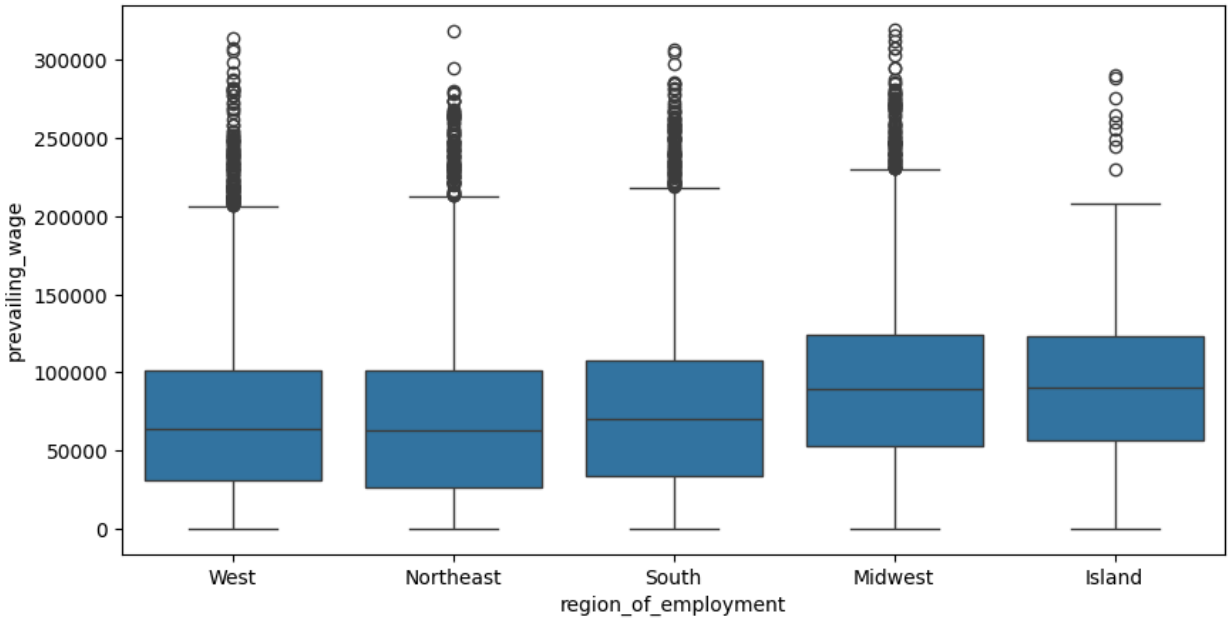


Fig 18: Observation on Prevailing_Wage and Region of Employment

Observations:

- Midwest and island have slightly higher median wages compared to other regions

Observation on Unit Of Wages and Case Status

case_status	Certified	Denied
unit_of_wage		
Hour	0.346314	0.653686
Month	0.617978	0.382022
Week	0.621324	0.378676
Year	0.698850	0.301150

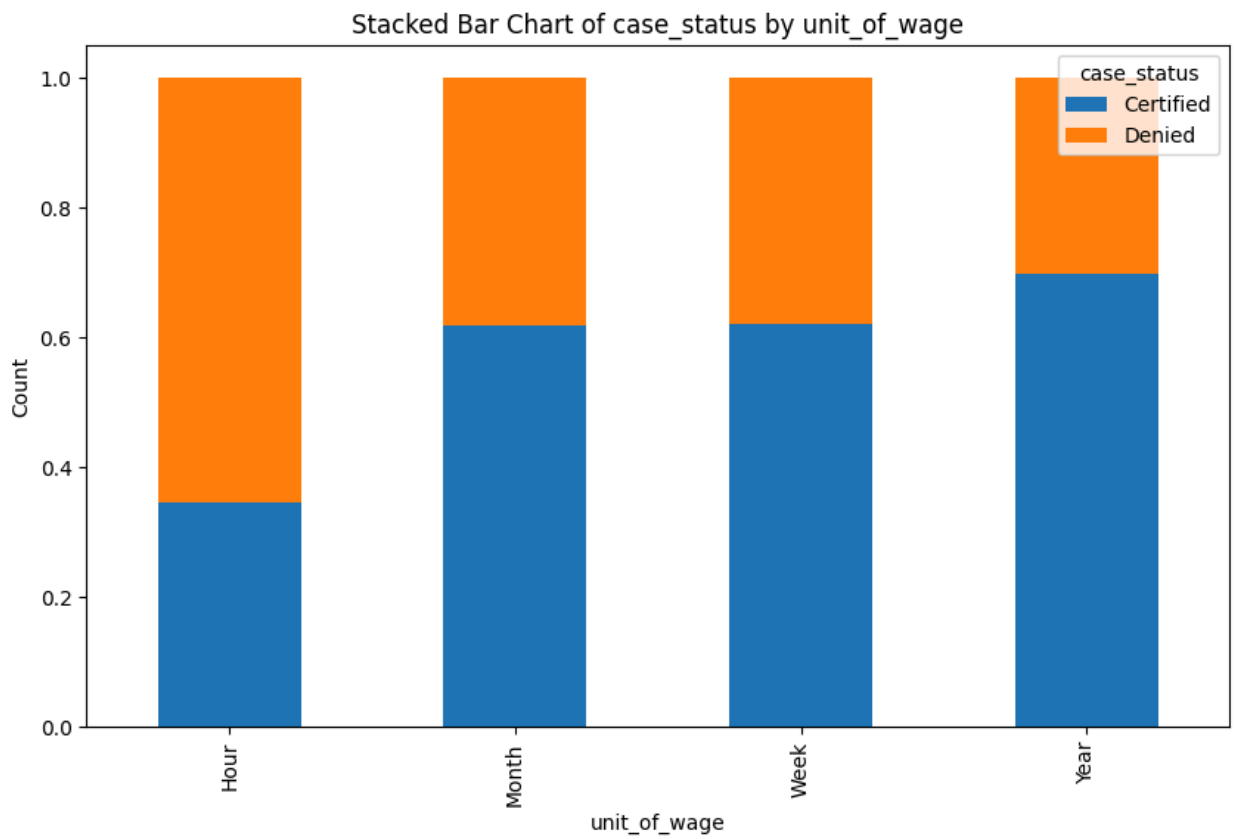


Fig 19: Observation on Unit Of Wages and Case Status

Observations:

- Yearly is most likely to be certified but week and month's percentage of employees certified is almost the same.

Observation on Job Training and Case Study

case_status	Certified	Denied
requires_job_training		
N	0.666459	0.333541
Y	0.678849	0.321151

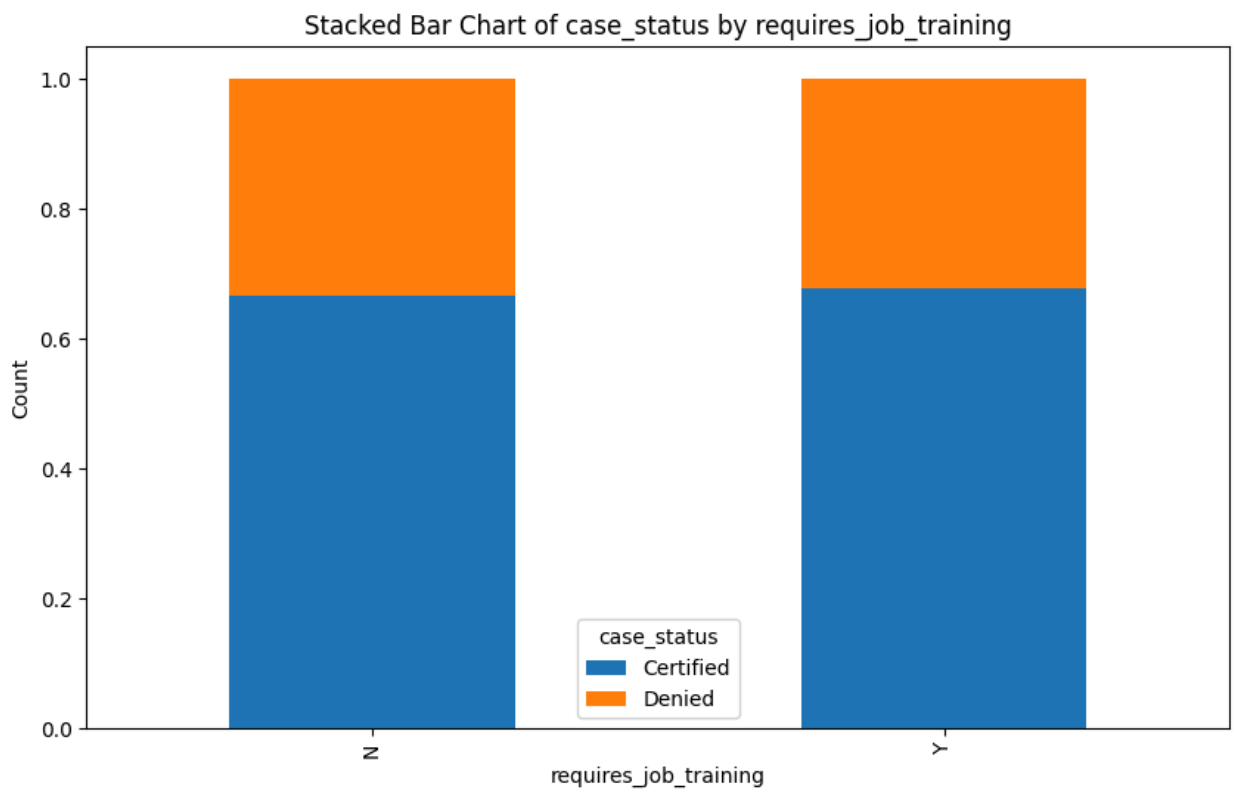


Fig 20: Observation on Job Training and Case Study

Observations:

- Job training requirements are not really affected per observation.

Observation on Position Type and Case Status

case_status	Certified	Denied
full_time_position		
Y	0.665832	0.334168
N	0.685260	0.314740

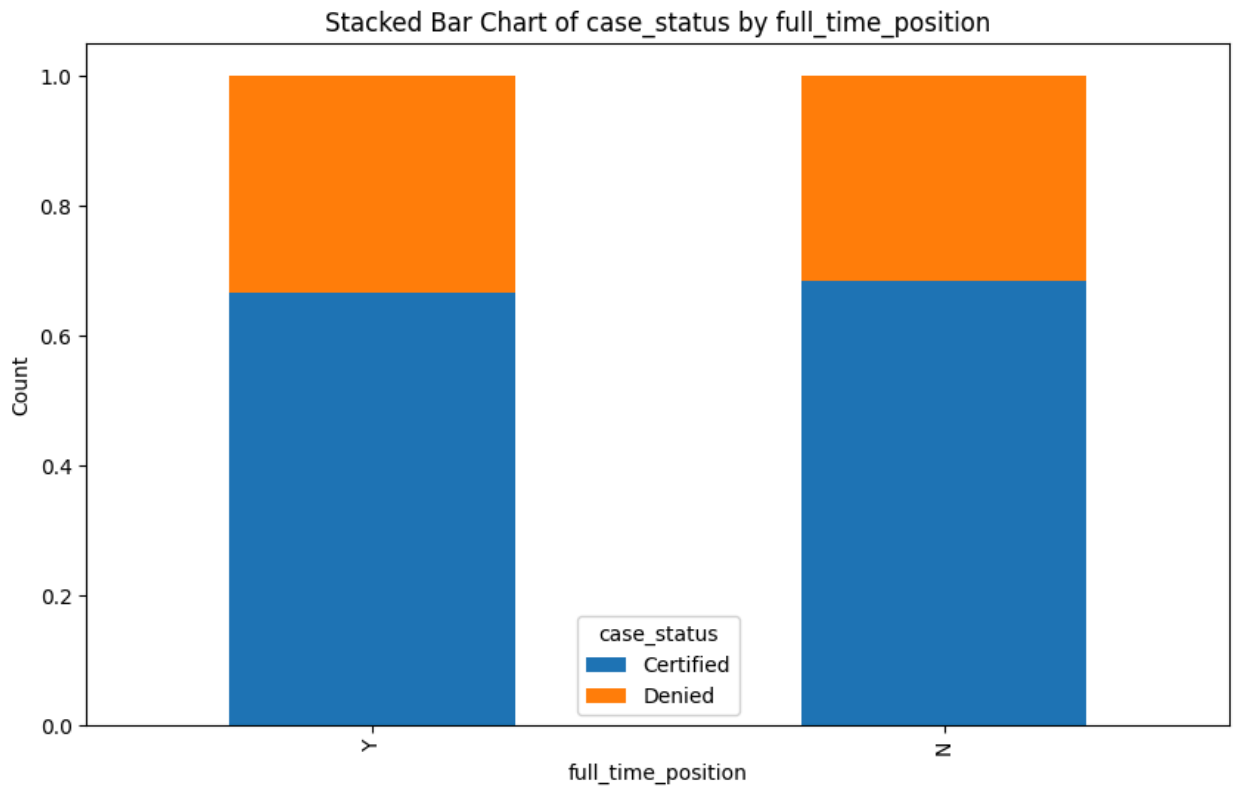


Fig 21: Observation on Position Type and Case Status

Observations:

- Observations show visa certification does not really impact whether it's part time or full time.

Data Processing

Data Preprocessing

- Missing value treatment (if needed)
- Feature engineering
- Outlier detection and treatment (if needed)
- Preparing data for modeling
- Any other preprocessing steps (if needed)

Missing value treatment

Observations:

- There are no missing values in the given dataset.

```
df.isnull().sum()
```

	0
continent	0
education_of_employee	0
has_job_experience	0
requires_job_training	0
no_of_employees	0
yr_of_estab	0
region_of_employment	0
prevailing_wage	0
unit_of_wage	0
full_time_position	0
case_status	0

dtype: int64

Fig 22: Checking Missing value

Feature Engineer

- Adding a new column, Yrs_snc_estab, including years since the establishment's final year is 2016.
- Dropping yr_of_estab from the dataframe.
- Creating a column including equivalent hourly wages.
- Dropped privilege_wage column from the dataframe.

```
df.sample(10, random_state=1)
```

	continent	education_of_employee	has_job_experience	requires_job_training	no_of_employees	region_of_employment	unit_of_wage	full_time_position	case_status	yrs_since_estab	hourly_wage
17639	Asia	Bachelor's	Y	N	567	Midwest	Year	Y	Certified	24	12.905245
23951	Oceania	Bachelor's	N	N	619	Midwest	Year	Y	Certified	78	31.932683
8625	Asia	Master's	N	N	2635	South	Hour	Y	Certified	11	887.292100
20206	Asia	Bachelor's	Y	Y	3184	Northeast	Year	Y	Certified	30	23.767212
7471	Europe	Bachelor's	Y	N	4681	West	Year	Y	Denied	88	23.973649
3433	Asia	Bachelor's	Y	N	222	South	Hour	Y	Certified	27	813.726100
24440	Europe	High School	N	Y	3278	South	Year	Y	Denied	22	98.532880
12104	Asia	Master's	Y	N	1359	West	Year	N	Certified	19	97.229346
15656	Asia	Bachelor's	N	N	2081	West	Year	Y	Denied	13	53.708183
23110	North America	Bachelor's	Y	N	854	Northeast	Hour	Y	Denied	18	444.825700

- Checked the statistical summary of the data set after performing feature engineering using **describe().**

```
df.describe().T
```

	count	mean	std	min	25%	50%	75%	max
no_of_employees	25480.0	5667.043210	22877.928848	-26.000000	1022.00000	2109.000000	3504.000000	602069.00000
yrs_since_estab	25480.0	36.590071	42.366929	0.000000	11.00000	19.000000	40.000000	216.00000
hourly_wage	25480.0	94.902995	278.176919	0.048077	22.64806	39.826663	60.012036	7004.39875

Observations:

- The oldest employer was established 216 years before the data was compiled.
- Median and mean values of yrs_snc_estab are ~19 years and 36.5, respectively.
- The average hourly wage is approximately 95.

Outlier detection and treatment

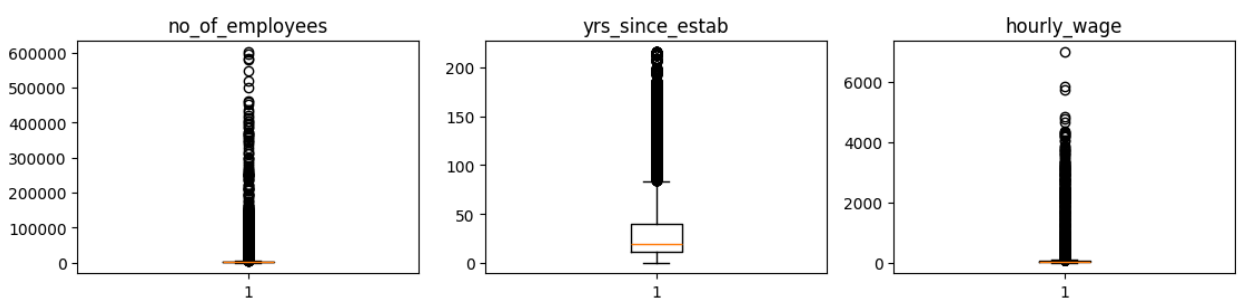


Fig 23: Outlier Detection

Observations:

Not all the outliers detected based on the 1.5-IQR rule are true outliers, but we will be removing the very large values.

Treating Outliers

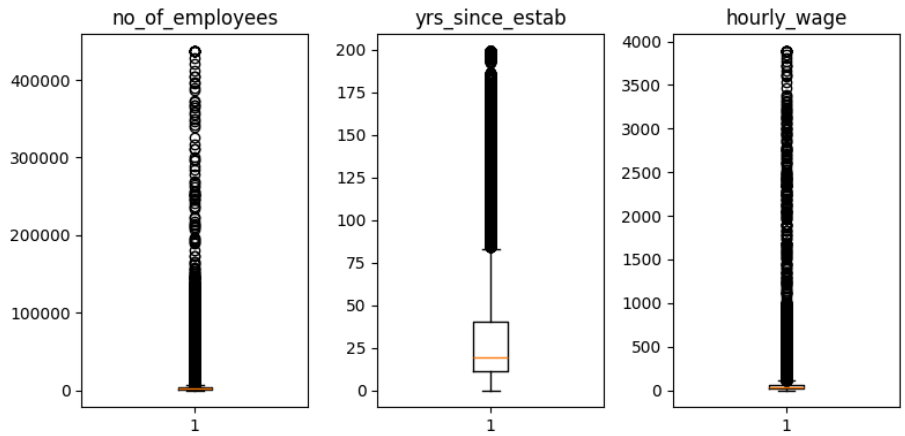


Fig 24: Treating Outliers

Preparing Data For Modeling

a. Before we proceed to build a model, we'll have to encode categorical features.
b. splitting the data into the dependent(y) and independent variables(X).
   i. case_status as a dependent variable(y).
   ii. All the remaining columns except case_status as independent variable(X).
c. Creating the dummy variables using **get_dummies()** for the columns having the category column.
d. Splitting the data in a 70:30 ratio for train and test data using **train_test_split()**.
e. The number of rows in train data is 17836.
f. The number of rows in test data is 7644.

Bagging - Model Building and Hyperparameter Tuning

Decision Tree Model

We will build our model using the DecisionTreeClassifier function, using default 'gini' criteria to split.

```
dtree =DecisionTreeClassifier(criterion="gini", random_state=1,)
dtree.fit(X_train, y_train)
```

```
▾ DecisionTreeClassifier
DecisionTreeClassifier(random_state=1)
```

Confusion Matrix of Decision Tree Classifier's Performance on Test Data

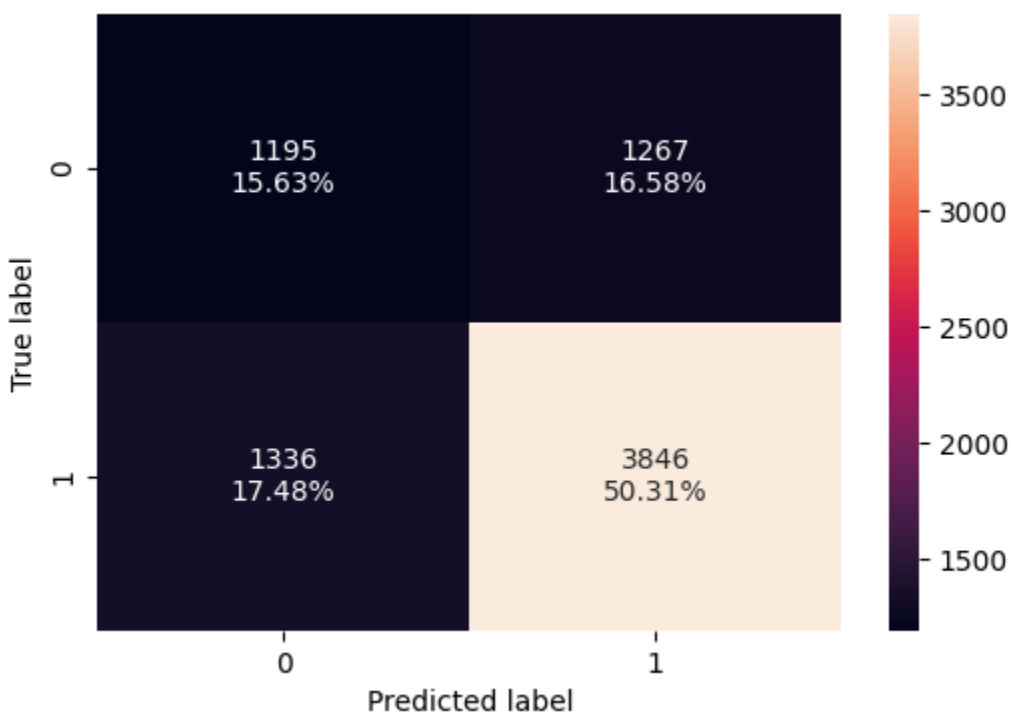


Fig 25: Confusion Matrix of Decision Tree Classifier's Performance on Test Data

The Performance of a Training and Test Data Of Decision Tree

Train Performance			
Accuracy	Recall	Precision	F1_Score
0.1	0.1	0.1	0.1
Test Performance			
Accuracy	Recall	Precision	F1_Score
0.659471	0.742184	0.7522	0.747159

Table 2: The Performance of a Training and Test Data Of Decision Tree

Observation:

• The decision tree model performs well for the training data set and all performance metrics, i.e., accuracy, recall, precision, and F1-score are 1.00.

• However, on the test set performance is not that good (F1-score is 0.75 approximately) indicating the model is overfitting. As a result, there is a need for hyperparameter tuning through grid search.

Decision Tree Classifier with Hyperparameter Tuning

```
DecisionTreeClassifier
DecisionTreeClassifier(max_depth=np.int64(4), max_leaf_nodes=np.int64(14),
                       min_impurity_decrease=0.0001, min_samples_split=50,
                       random_state=1)
```

Confusion Matrix of Tuned Decision Tree Classifier's Performance on Test Data

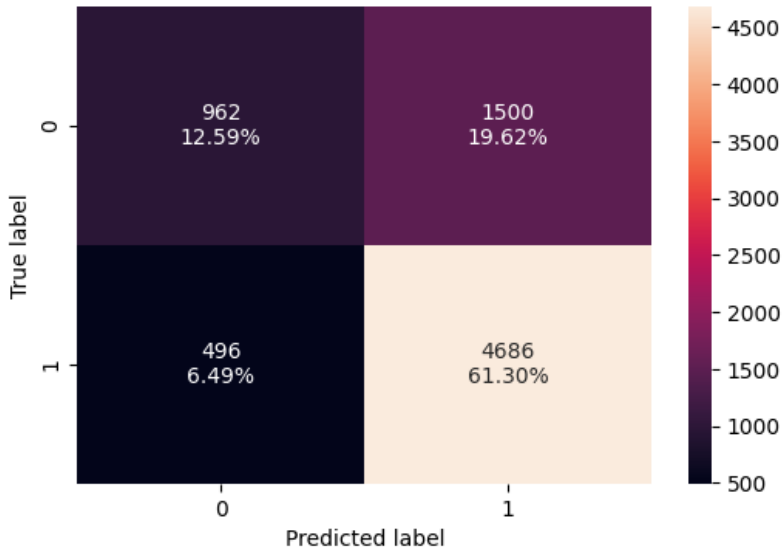


Fig 26: Confusion Matrix of Tuned Decision Tree Classifier's Performance on Test Data

The Tuned Decision Tree Classifier's Performance of a Training and Test Data

Train Performance			
Accuracy	**Recall**	**Precision**	**F1_Score**
0.731778	**0.907317**	**0.74442**	**0.817836**
Test Performance			
Accuracy	**Recall**	**Precision**	**F1_Score**
0..73888	**0.904284**	**0.757517**	**0.824419**

Table 3: The Tuned Decision Tree Classifier's Performance of Training and Testing Data

Observation:

• The turned decision tree model performs equally well overall on the training and testing datasets, indicating that the model is not overfitting.

• F1-score for the test set has been increased from approximately 0.75 for the initial model to 0.82 approximately for the tuned model.

Feature Importance of Tuned Decision True Model

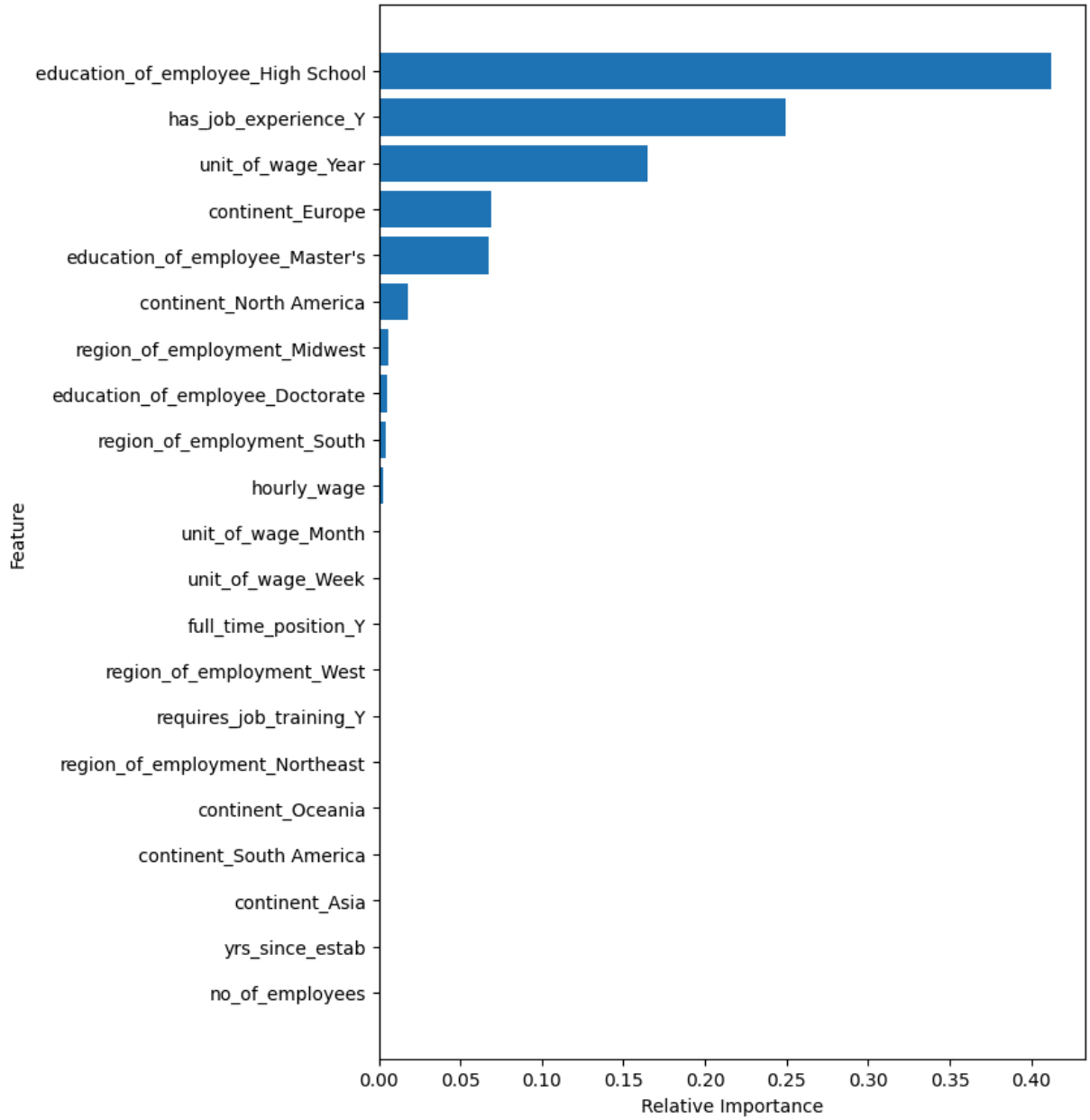


Fig 27: Feature Importance of Tuned Decision True Model

Observation:

• Four top independent variables of importance in the tuned decision tree model are education_of_employee_high_school, has_job_experience, unit_of_wage_year, and continental Europe.

Bagging Classifier

We will build our model using the BaggingClassifier function

```
BaggingClassifier
BaggingClassifier(random_state=1)
```

Confusion Matrix of Bagging Classifier's Performance on Test Data

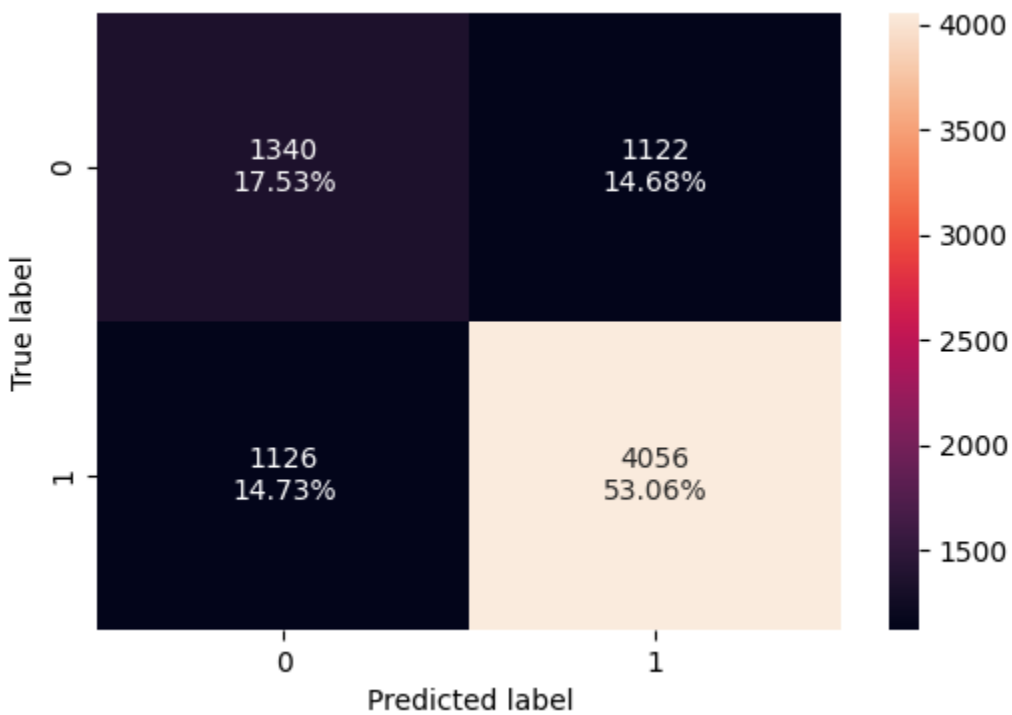


Fig 28: Confusion Matrix of Bagging Classifier's Performance on Test Data

The Performance of a Training and Test Data Of Bagging Classifier

Train Performance			
Accuracy	Recall	Precision	F1_Score
0.985311	0.986397	0.991423	0.988904
Test Performance			
Accuracy	Recall	Precision	F1_Score
0.705913	0.782709	0.783314	0.783012

Table 4: The Performance of a Training and Test Data Of Bagging Classifier

Observations:

- The model has a little better performance on the test data set compared to the initial decision tree not turned model.
- Looking into the high-performance metrics for the training data set, the model is overfitting and needs hyperparameter tuning.

Bagging Classifier with Hyperparameter Tuning

```
BaggingClassifier
BaggingClassifier(max_features=0.7, max_samples=0.7, n_estimators=60,
                  random_state=1)
```

Confusion Matrix of Tuned Bagging Classifier's Performance on Test Data

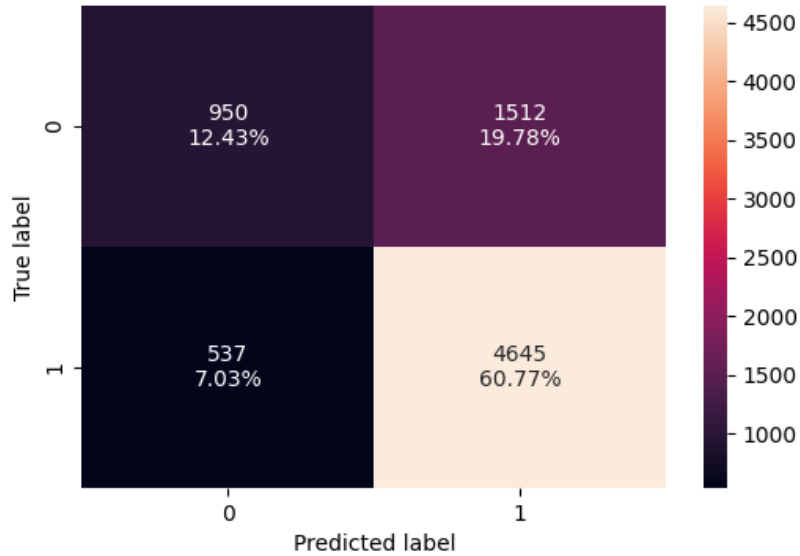


Fig 29: *Confusion Matrix of Tuned Bagging Classifier's Performance on Test Data*

The Tuned Bagging Classifier's Performance of a Training and Test Data

Train Performance			
Accuracy	**Recall**	**Precision**	**F1_Score**
0.998879	**1.0**	**0.998313**	**0.999156**
Test Performance			
Accuracy	**Recall**	**Precision**	**F1_Score**
0..731947	**0.896372**	**0.754426**	**0.819296**

Table 5: The Tuned Bagging Classifier's Performance of Training and Testing Data

Observations:

- The tuned model's performance on the test data set has been improved a bit compared to the initial bagging model - the F1-score has been increased from 0.78 for the initial model to 0.82 for the tuned model.
- The model seems to be overfitting in the training data.

Random Forest Classifier

We will build our model using the random Forest Classifier function

```
RandomForestClassifier
RandomForestClassifier(random_state=1)
```

Confusion Matrix of Random Forest Classifier's Performance on Test Data

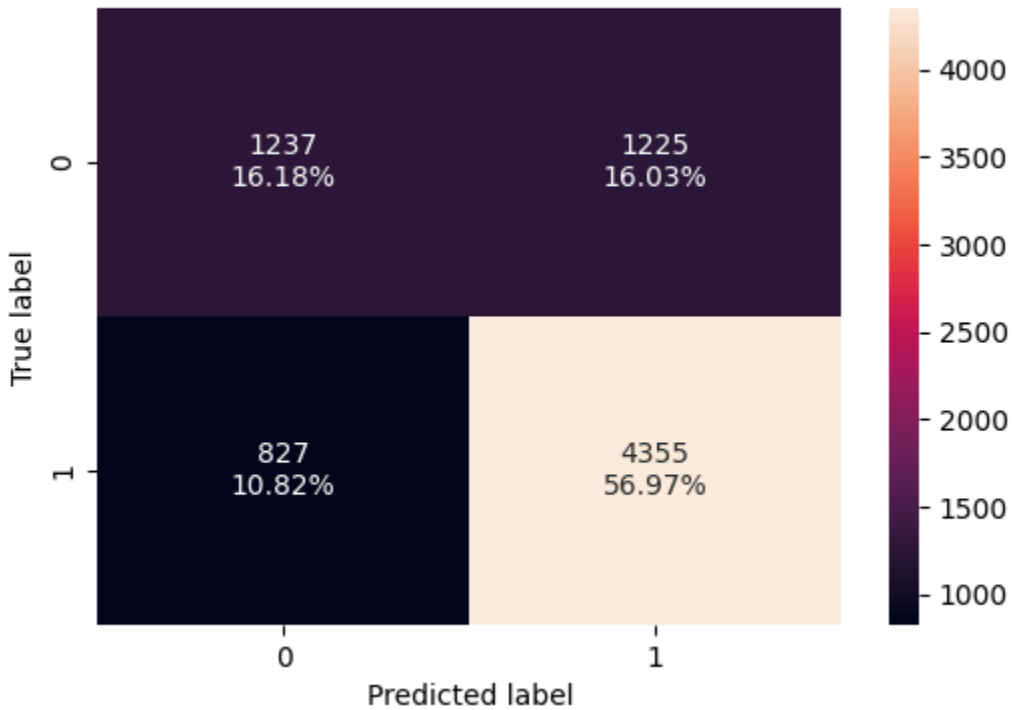


Fig 30: Confusion Matrix of Random Forest Classifier's Performance on Test Data

The Performance of a Training and Test Data Of Random Forest Classifier

Train Performance			
Accuracy	Recall	Precision	F1_Score
0.999944	0.986397	0.999916	0.999958
Test Performance			
Accuracy	Recall	Precision	F1_Score
0.731554	0.840409	0.780466	0.809329

Table 6: The Performance of a Training and Test Data Of Random Forest Classifier

Observations:

- Similar to the models above, this Random Forest is also overfit to the training dataset but has the highest F1 score so far.

Random Forest Classifier with Hyperparameter Tuning

```
RandomForestClassifier
RandomForestClassifier(max_depth=np.int64(4), max_samples=1.0,
                       min_samples_split=200, n_estimators=np.int64(80),
                       oob_score=True, random_state=1)
```

Confusion Matrix of Tuned Random Forest Classifier's Performance on Test Data

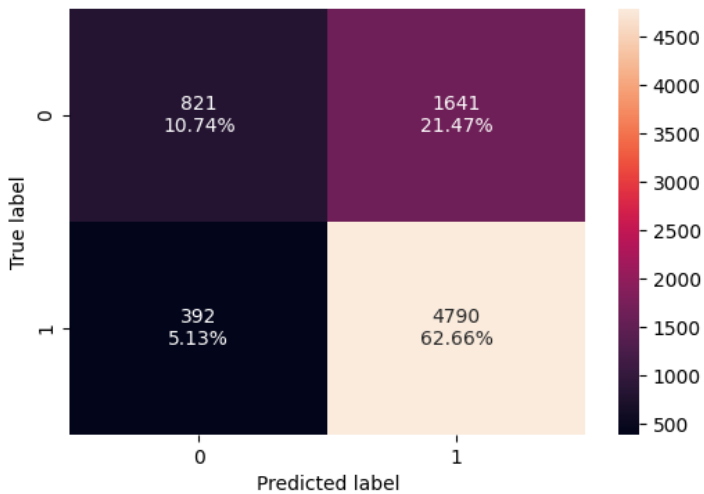


Fig 31: Confusion Matrix of Tuned Random Forest Classifier's Performance on Test Data

The Tuned Random Forest Classifier Performance of a Training and Test Data

Train Performance			
Accuracy	Recall	Precision	F1_Score
0.729984	0.927171	0.735129	0.820057
Test Performance			
Accuracy	Recall	Precision	F1_Score
0.73404	0.924354	0.74483	0.824938

Table 7: The Tuned Random Forest Performance of a Training and Test Data

Observations:

- *Observation shows performance metrics are close for both training and test data sets, showing the model is no longer overfitting.*

Feature Importance of Tuned Random Forest Classifier

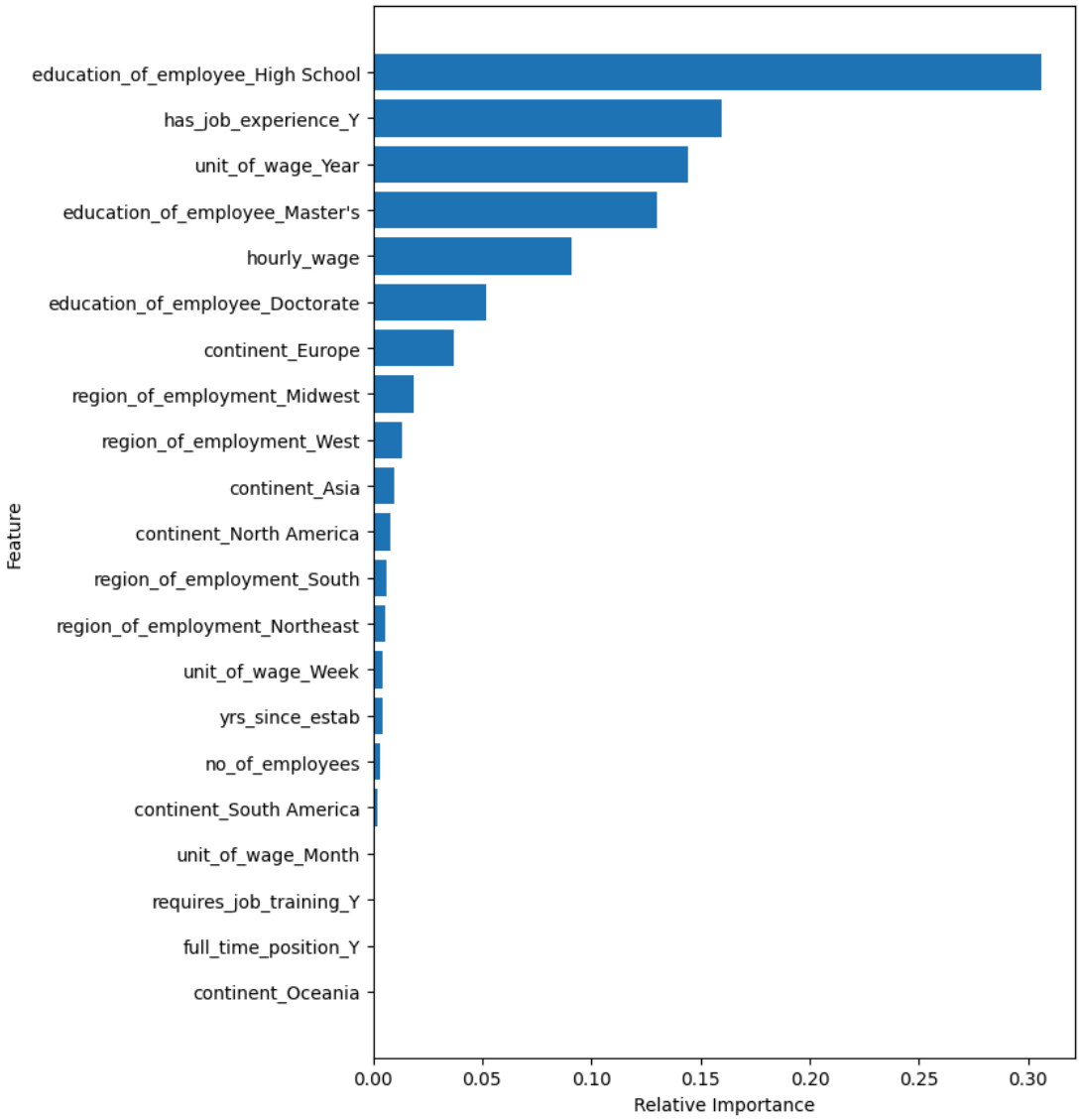


Fig 32: Feature Importance of Tuned Random Forest Classifier

Observations:

- Top independent features of importance in the tuned random forest mode areducation_of_employee_high_school, has_job_experience_Y, unit_of_wage_year, and education_of_the_emoloyee_masters.

Boosting - Model Building and Hyperparameter Tuning

AdaBoost Classifier

We will build our model using the AdaBoostClassifier function

```
AdaBoostClassifier
AdaBoostClassifier(random_state=1)
```

Confusion Matrix of AdaBoost Classifier's Performance on Test Data

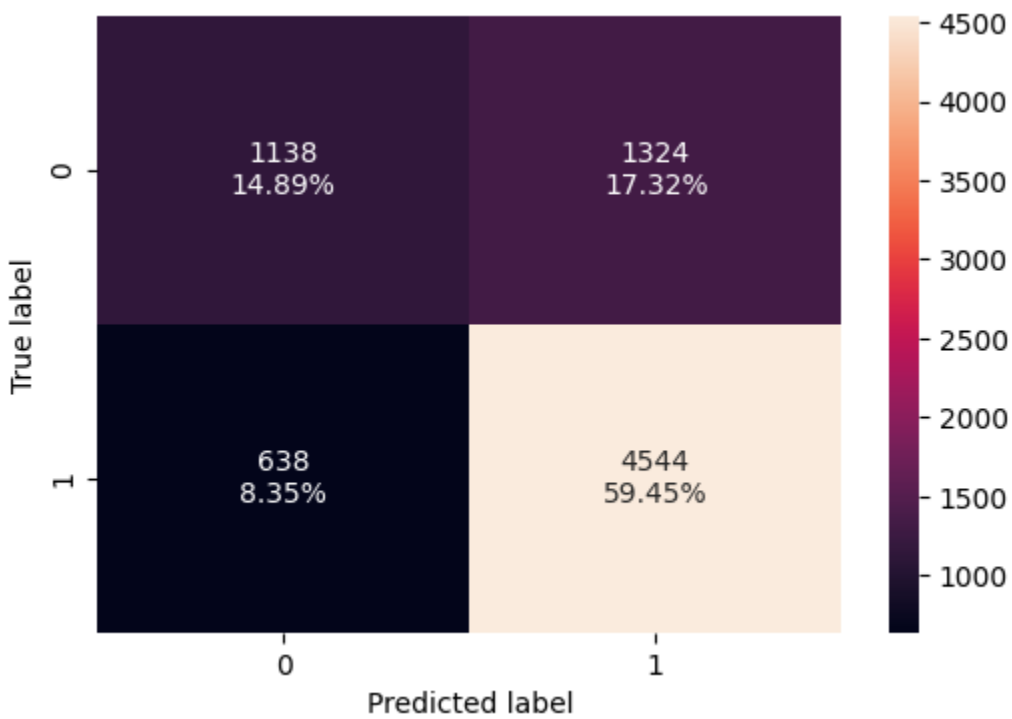


Fig 33: Confusion Matrix of Tuned AdaBoost Classifier's Performance on Test Data

The Performance of a Training and Test Data Of AdaBoost Classifier

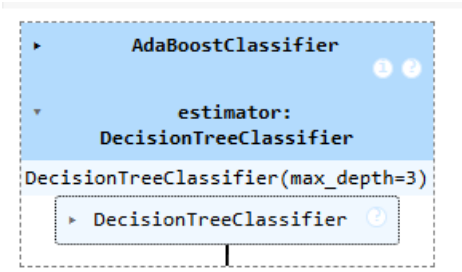
Train Performance			
Accuracy	Recall	Precision	F1_Score
0.738731	0.877746	0.763785	0.816809
Test Performance			
Accuracy	Recall	Precision	F1_Score
0.743328	0.876882	0.774369	0.822443

Table 8: The Performance of a Training and Test Data Of AdaBoost Classifier

Observations:

- This AdaBoost classifier performs equally well on the training and testing datasets, indicating that the model is not overfit.
- Hyperparameter tuning may help to improve the model's performance as the metrics for the training and test data sets are very close.

AdaBoost Classifier with Hyperparameter Tuning



Confusion Matrix of Tuned AdaBoost Classifier's Performance on Test Data

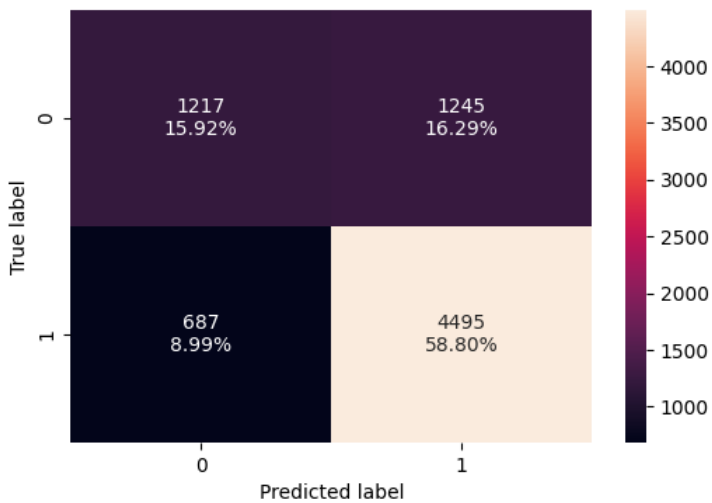


Fig 34: The Performance of a Training and Test Data Of Tuned AdaBoost Classifier

The Tuned AdaBoost Classifier Performance of a Training and Test Data

Train Performance			
Accuracy	Recall	Precision	F1_Score
0.745907	0.872423	0.773599	0.820044
Test Performance			
Accuracy	Recall	Precision	F1_Score
0.747253	0.867426	0.783101	0.823109

Table 9: The Tuned AdaBoost Classifier Performance of a Training and Test Data

Observations:

- *Significant changes were not observed in the model performance after tuning*

Feature Importance of Tuned Random Forest Classifier

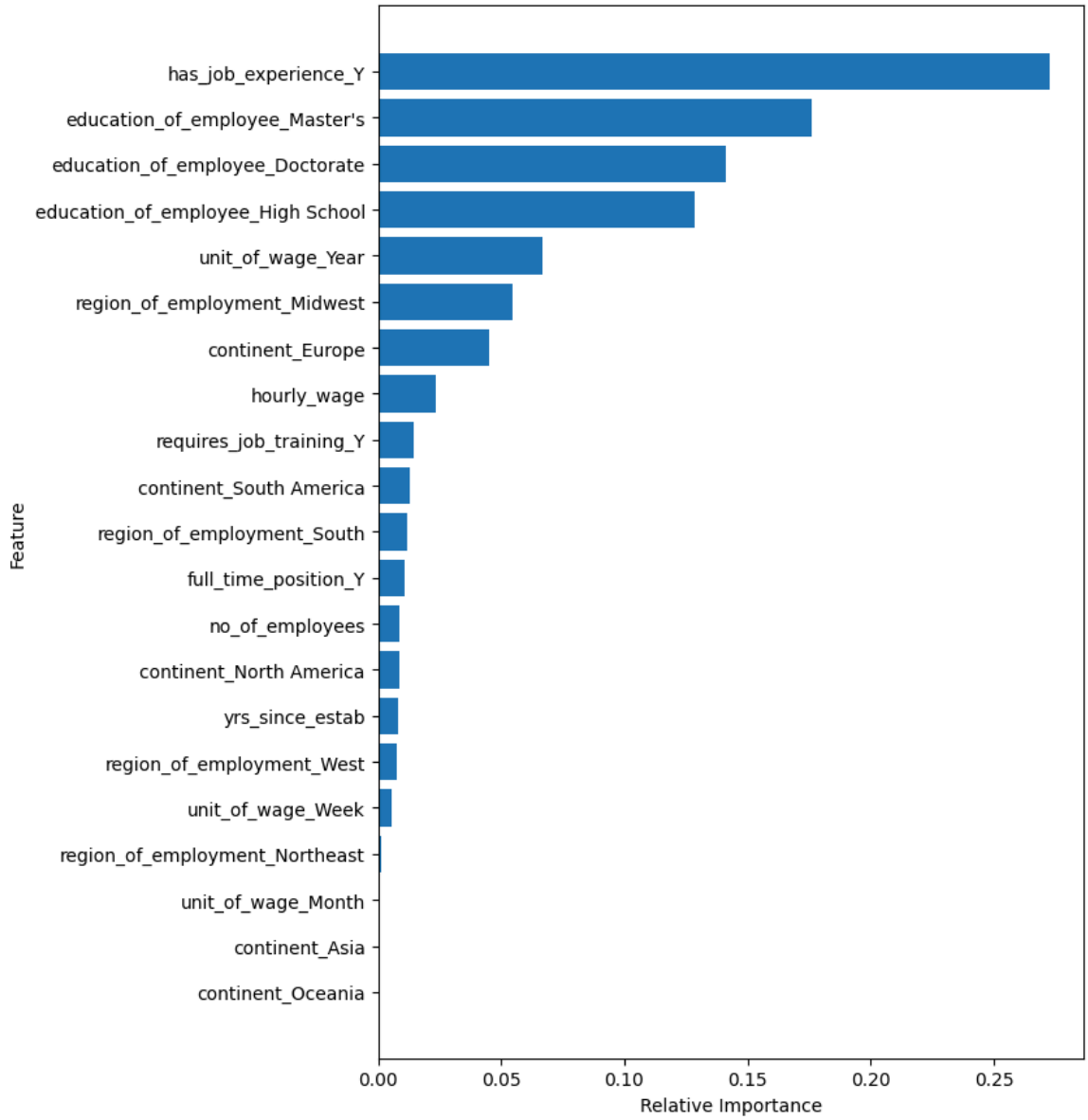


Fig 35: Feature Importance of Tuned AdaBoost

Observations:

- The top three independent features of importance in the tuned AdaBoost model are has_job_experience_Y, education_of_employee_Master's, education_of_employee_Doctorate.

Gradient Boosting Classifier

We will build our model using the AdaBoostClassifier function

```
GradientBoostingClassifier
GradientBoostingClassifier(random_state=1)
```

Confusion Matrix of Gradient Boosting Classifier's Performance on Test Data

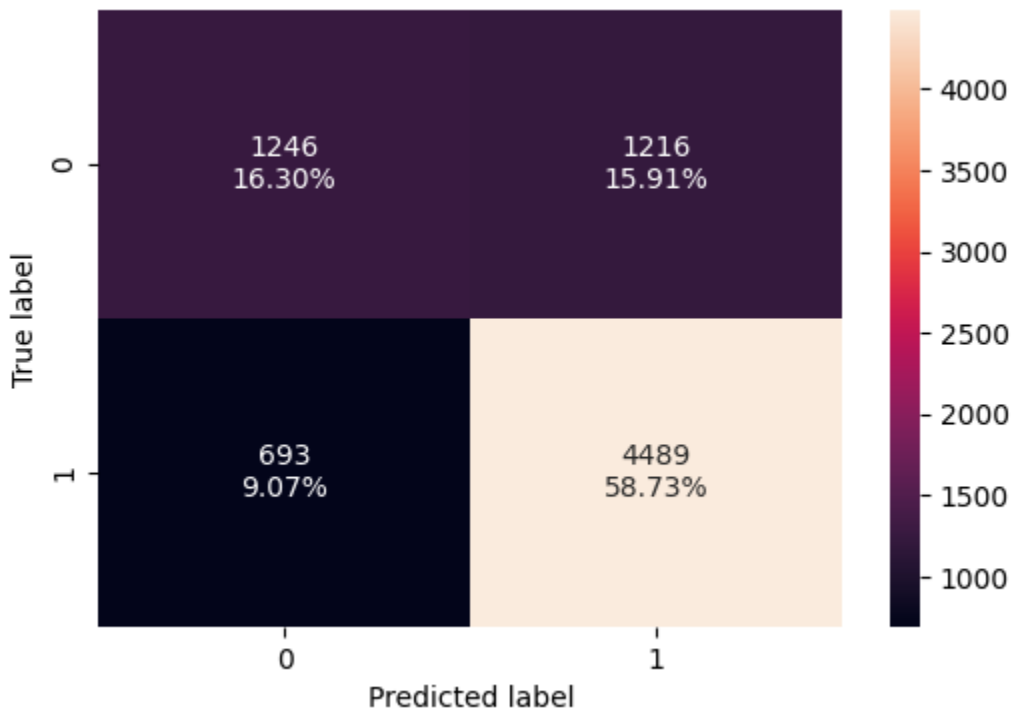


Fig 36: Confusion Matrix of Gradient Boosting Classifier's Performance on Test Data

The Performance of a Training and Test Data Of Gradient Boost Classifier

Train Performance			
Accuracy	Recall	Precision	F1_Score
0.755326	0.874789	0.782261	0.825941
Test Performance			
Accuracy	Recall	Precision	F1_Score
0.750262	0.866268	0.786854	0.824653

Table 10: The Performance of a Training and Test Data Of Gradient Boost Classifier

Observations:

- *Both the training and test data sets do not show overfitting.*
- *The F1-score for both training and test data sets is above 0.82, which looks good.*

Gradient Boost Classifier with Hyperparameter Tuning

```
GradientBoostingClassifier
init: AdaBoostClassifier
AdaBoostClassifier(random_state=1)
AdaBoostClassifier
```

Confusion Matrix of Tuned Gradient Boost Classifier's Performance on Test Data

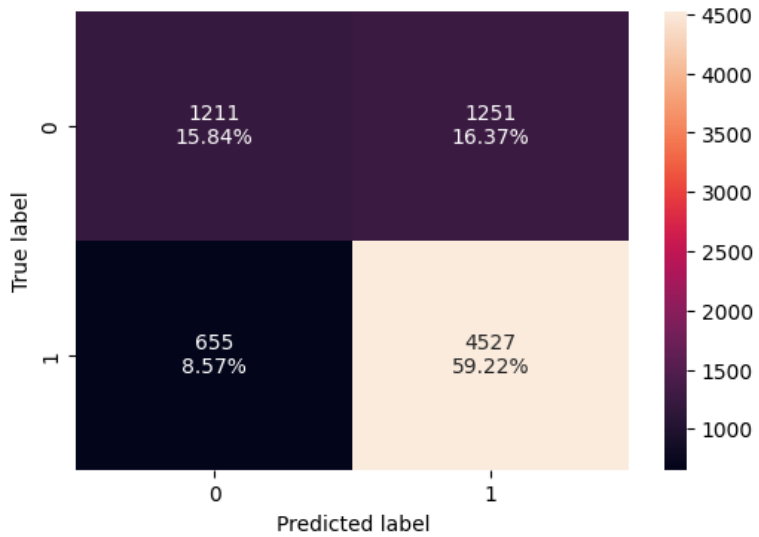


Fig 37: Confusion Matrix of Tuned Gradient Boost Classifier's Performance on Test Data

The Tuned Gradient Boost Classifier Performance of a Training and Test Data

Train Performance			
Accuracy	**Recall**	**Precision**	**F1_Score**
0.750056	**0.878422**	**0.77497**	**0.82346**
Test Performance			
Accuracy	**Recall**	**Precision**	**F1_Score**
0.750654	**0.873601**	**0.783489**	**0.826095**

Table 11: The Tuned Gradient Boost Classifier Performance of a Training and Test Data

Observations:

- Hyperparameter tuning slightly improves the performance of the gradient boosting model.

Feature Importance of Tuned Gradient Boost Classifier

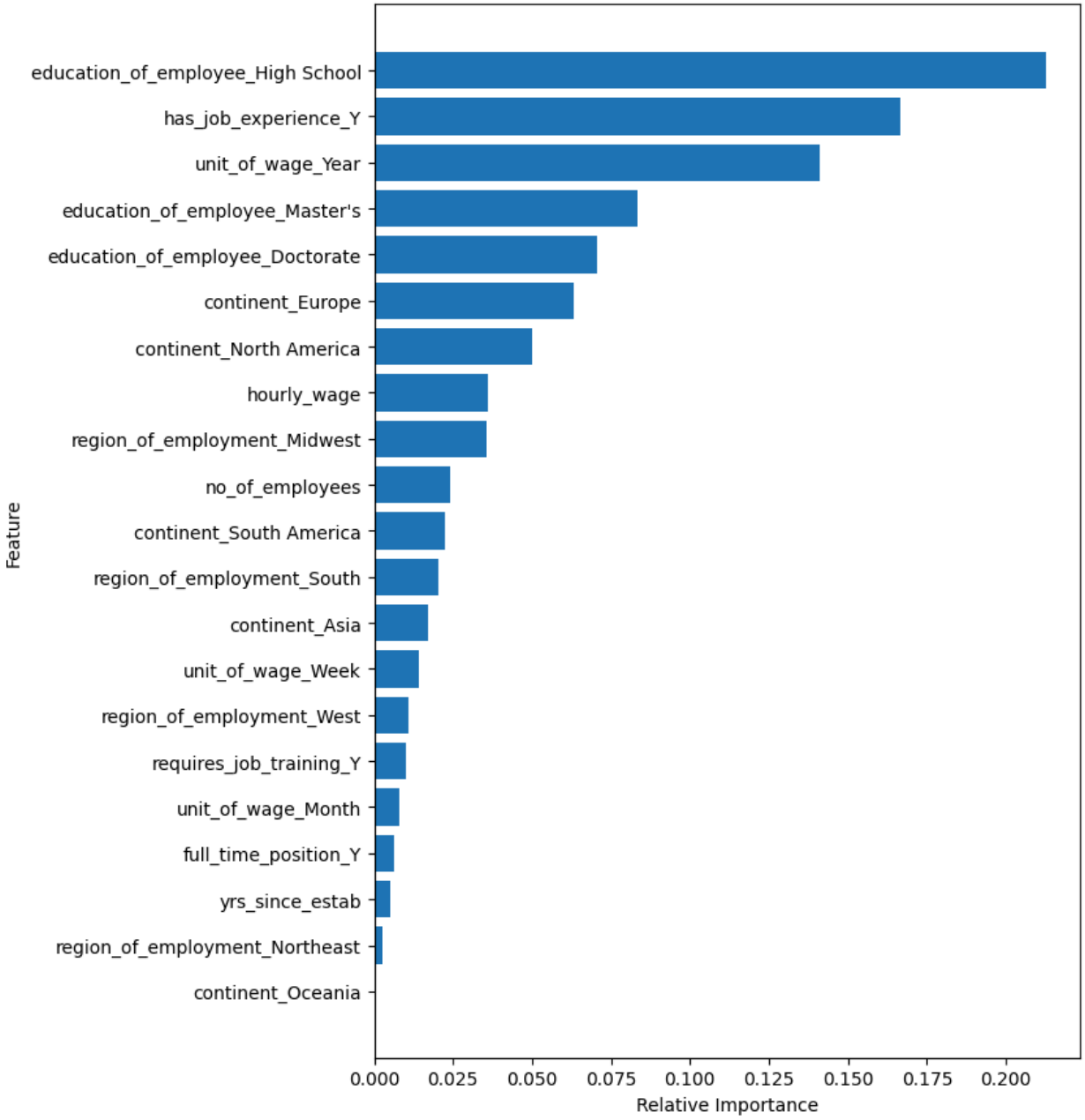


Fig 38: Feature Importance of Tuned Gradient Boost Classifier

Observations:

- The top three independent features of importance in the tuned gradient boosting model are education_of_employee_high_school, has_job_experience, unit_of_wage_year.

XGBoost Classifier

We will build our model using the XGBoostClassifier function

```
xg_boost.fit(X_train, y_train)
```

```
XGBClassifier
XGBClassifier(base_score=None, booster=None, callbacks=None,
              colsample_bylevel=None, colsample_bynode=None,
              colsample_bytree=None, device=None, early_stopping_rounds=None,
              enable_categorical=False, eval_metric='logloss',
              feature_types=None, feature_weights=None, gamma=None,
              grow_policy=None, importance_type=None,
              interaction_constraints=None, learning_rate=None, max_bin=None,
              max_cat_threshold=None, max_cat_to_onehot=None,
              max_delta_step=None, max_depth=None, max_leaves=None,
              min_child_weight=None, missing=nan, monotone_constraints=None,
              multi_strategy=None, n_estimators=None, n_jobs=None,
```

Confusion Matrix of XGBoosting Classifier's Performance on Test Data

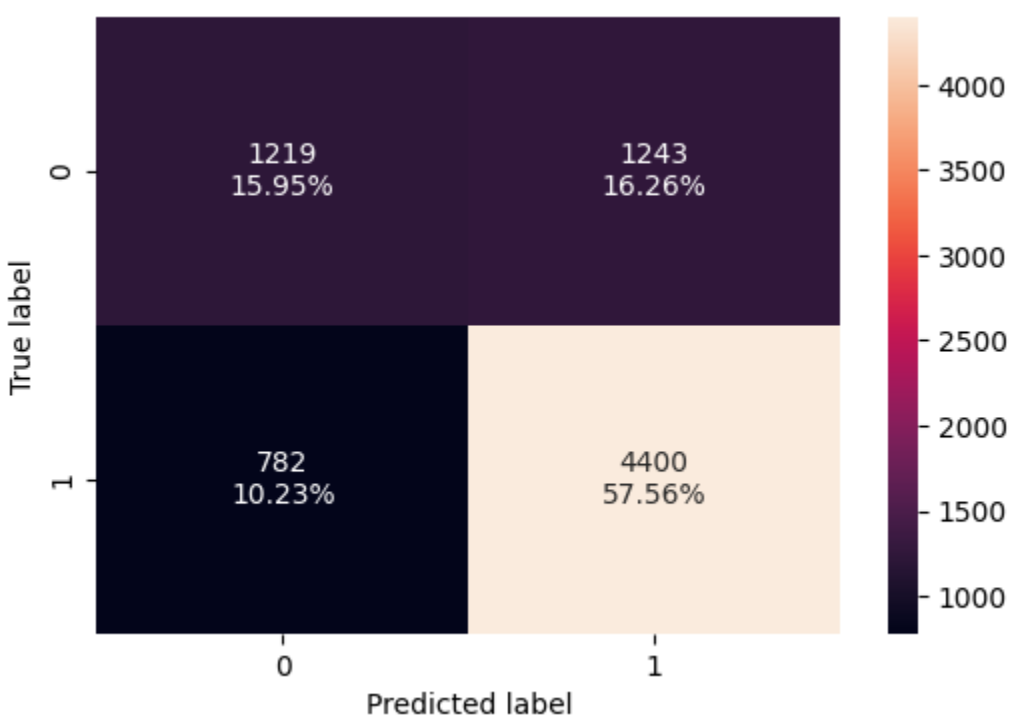


Fig 39: Confusion Matrix of XGBoosting Classifier's Performance on Test Data

The Performance of a Training and Test Data Of XGBoost Classifier

Train Performance			
Accuracy	Recall	Precision	F1_Score
0.837295	0.928439	0.842456	0.88336
Test Performance			
Accuracy	Recall	Precision	F1_Score
0.735086	0.849093	0.779727	0.812933

Table 12: The Performance of a Training and Test Data Of XGBoost Classifier

Observations:

- This XGBoost classifier performs equally well on the training and testing datasets, indicating that the model is not overfit.
- Hyperparameter tuning could be used to see if there will be any improvement with the model as it looks slightly overfitting.

XG Boost Classifier with Hyperparameter Tuning

```
XGBClassifier
XGBClassifier(base_score=None, booster=None, callbacks=None,
              colsample_bylevel=0.6, colsample_bynode=None,
              colsample_bytree=0.8, device=None, early_stopping_rounds=None,
              enable_categorical=False, eval_metric='logloss',
              feature_types=None, feature_weights=None, gamma=3,
              grow_policy=None, importance_type=None,
              interaction_constraints=None, learning_rate=0.1, max_bin=None,
              max_cat_threshold=None, max_cat_to_onehot=None,
              max_delta_step=None, max_depth=None, max_leaves=None,
              min_child_weight=None, missing=nan, monotone_constraints=None,
              multi_strategy=None, n_estimators=50, n_jobs=None,
```

Confusion Matrix of Tuned XG Boost Classifier's Performance on Test Data

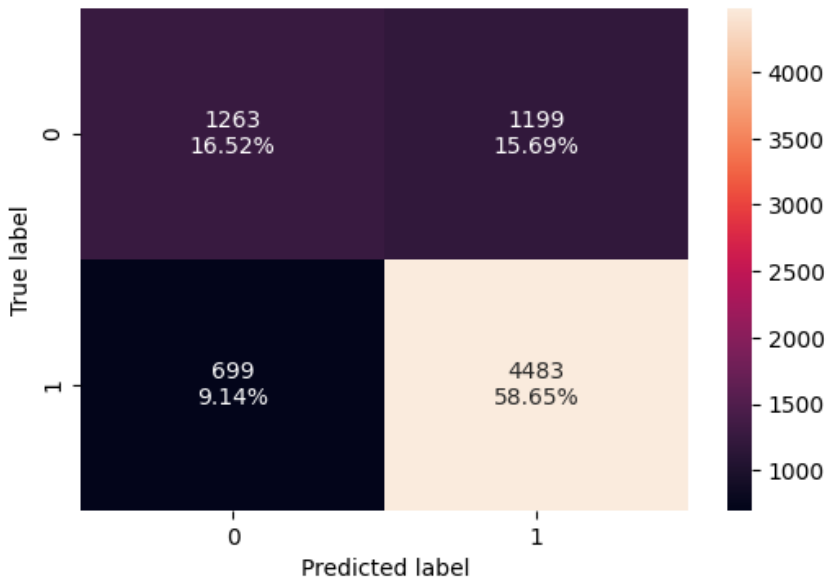


Fig 40: Confusion Matrix of Tuned XG Boost Classifier's Performance on Test Data

The Tuned XG Boost Classifier Performance of a Training and Test Data

Train Performance			
Accuracy	Recall	Precision	F1_Score
0.75841	0.874958	0.785438	0.827785
Test Performance			
Accuracy	Recall	Precision	F1_Score
0.751701	0.86511	0.788983	0825295

Table 13: The Tuned XG Boost Classifier Performance of a Training and Test Data

Observations:

- The model performance on the test set was improved a bit after tuning, increasing the F1-score from 0.812 to 0.825. The Tuned XGBoost model provides close performances on both the test and training data sets.

Feature Importance of Tuned Gradient Boost Classifier

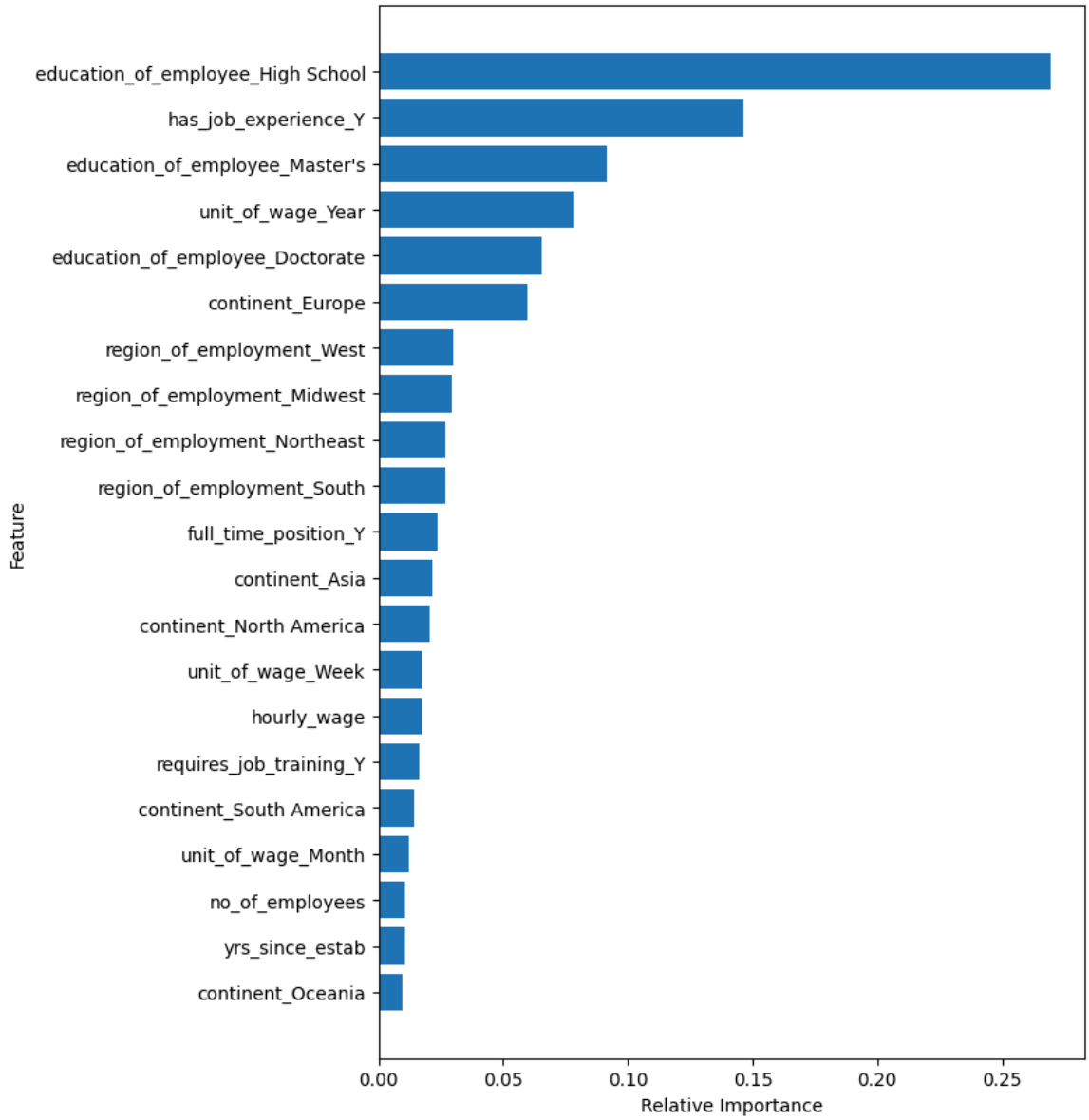


Fig 41: Feature Importance of Tuned Gradient Boost Classifier

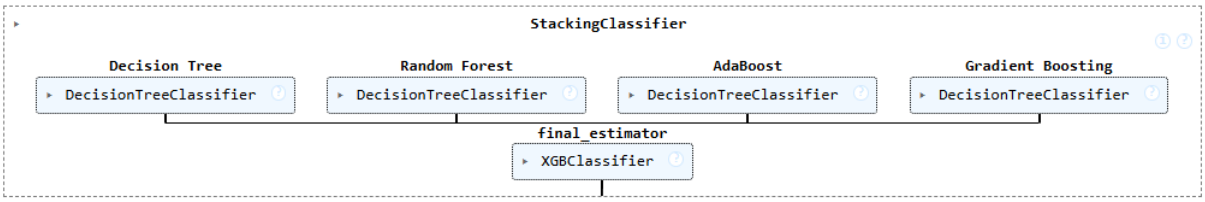
Observations:

- Top three independent features of importance in the tuned XGBoost model are education_of_employee_high_school, has_job_experience_Y and education_of_employee_Master's.

Stacking Classifier

We will build our model using the StackingClassifier function

```
stacking.fit(X_train, y_train)
```



Confusion Matrix of StackingClassifier's Performance on Test Data

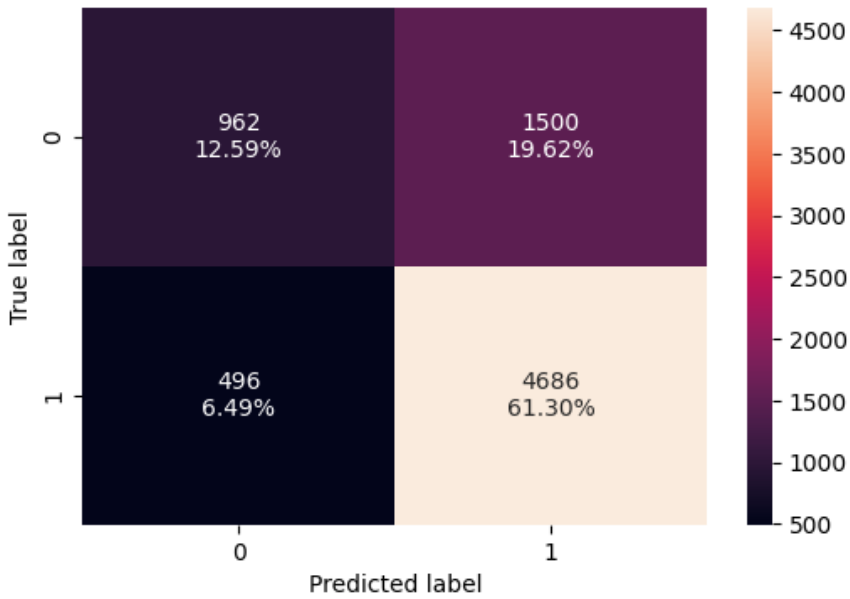


Fig 42: Confusion Matrix of Stacking Classifier's Performance on Test Data

The Performance of a Training and Test Data Of Stacking Classifier

Train Performance			
Accuracy	Recall	Precision	F1_Score
0.837295	0.928439	0.842456	0.88336
Test Performance			
Accuracy	Recall	Precision	F1_Score
0.837295	0.928439	0.842456	0.88336

Table 14: The Performance of a Training and Test Data Of Stacking Classifier

Observations:

- Almost the same performance to the tuned XGBoost in terms of all metrics.

Model Performance Comparison and Conclusions On Bagging and Boosting

Base models comparison

Training performance comparison of base models:

	Decision Tree	Bagging Classifier	Random Forest	AdaBoost Classifier	Gradient Boosting Classifier	XGBoost Classifier
Accuracy	1.0	0.985311	0.999944	0.738731	0.755326	0.837295
Recall	1.0	0.986397	1.000000	0.877746	0.874789	0.928439
Precision	1.0	0.991423	0.999916	0.763785	0.782261	0.842456
F1	1.0	0.988904	0.999958	0.816809	0.825941	0.883360

Testing performance comparison of base models:

	Decision Tree	Bagging Classifier	Random Forest	AdaBoost Classifier	Gradient Boosting Classifier	XGBoost Classifier
Accuracy	0.659471	0.705913	0.731554	0.743328	0.750262	0.735086
Recall	0.742184	0.782709	0.840409	0.876882	0.866268	0.849093
Precision	0.752200	0.783314	0.780466	0.774369	0.786854	0.779727
F1	0.747159	0.783012	0.809329	0.822443	0.824653	0.812933

Fig 43: Base models comparison

Tuned models comparison

Training performance comparison of tuned models:

	Tuned Decision Tree	Tuned Bagging Classifier	Tuned Random Forest	Tuned AdaBoost Classifier	Tuned Gradient Boosting Classifier	Tuned XGBoost Classifier
Accuracy	0.731778	0.998879	0.729984	0.745907	0.750056	0.758410
Recall	0.907317	1.000000	0.927171	0.872423	0.878422	0.874958
Precision	0.744420	0.998313	0.735129	0.773599	0.774970	0.785438
F1	0.817836	0.999156	0.820057	0.820044	0.823460	0.827785

Testing performance comparison of tuned models:

	Tuned Decision Tree	Tuned Bagging Classifier	Tuned Random Forest	Tuned AdaBoost Classifier	Tuned Gradient Boosting Classifier	Tuned XGBoost Classifier
Accuracy	0.738880	0.731947	0.734040	0.747253	0.750654	0.751701
Recall	0.904284	0.896372	0.924354	0.867426	0.873601	0.865110
Precision	0.757517	0.754426	0.744830	0.783101	0.783489	0.788983
F1	0.824419	0.819296	0.824938	0.823109	0.826095	0.825295

Fig 44: Tuned models comparison

Model Performance Comparison

Comparing the performance of all the model on training data

	Accuracy	Recall	Precision	F1
Decision Tree	1.000000	1.000000	1.000000	1.000000
Random Forest	0.999944	1.000000	0.999916	0.999958
Tuned Bagging Classifier	0.998879	1.000000	0.998313	0.999156
Bagging Classifier	0.985311	0.986397	0.991423	0.988904
XGBoost Classifier	0.837295	0.928439	0.842456	0.883360
Tuned XGBoost Classifier	0.758410	0.874958	0.785438	0.827785
Gradient Boosting Classifier	0.755326	0.874789	0.782261	0.825941
Tuned Gradient Boosting Classifier	0.750056	0.878422	0.774970	0.823460
Tuned Random Forest	0.729984	0.927171	0.735129	0.820057
Tuned AdaBoost Classifier	0.745907	0.872423	0.773599	0.820044
Tuned Decision Tree	0.731778	0.907317	0.744420	0.817836
AdaBoost Classifier	0.738731	0.877746	0.763785	0.816809

Fig 45: Comparing the performance of all the model on training data

Comparing the performance of all the model on testing data

	Accuracy	Recall	Precision	F1
Tuned Gradient Boosting Classifier	0.750654	0.873601	0.783489	0.826095
Tuned XGBoost Classifier	0.751701	0.865110	0.788983	0.825295
Tuned Random Forest	0.734040	0.924354	0.744830	0.824938
Gradient Boosting Classifier	0.750262	0.866268	0.786854	0.824653
Tuned Decision Tree	0.738880	0.904284	0.757517	0.824419
Tuned AdaBoost Classifier	0.747253	0.867426	0.783101	0.823109
AdaBoost Classifier	0.743328	0.876882	0.774369	0.822443
Tuned Bagging Classifier	0.731947	0.896372	0.754426	0.819296
XGBoost Classifier	0.735086	0.849093	0.779727	0.812933
Random Forest	0.731554	0.840409	0.780466	0.809329
Bagging Classifier	0.705913	0.782709	0.783314	0.783012
Decision Tree	0.659471	0.742184	0.752200	0.747159

Fig 46: Comparing the performance of all the model on testing data

Observations:

- On the training data, the Decision Tree achieved the highest performance (F1 = 1.0), indicating it has perfectly fit the training set. However, this also suggests overfitting, as such perfect scores are rarely sustainable on unseen data.
- On the testing data, the Tuned Gradient Boosting Classifier achieved the best F1 score (≈0.826), making it the most reliable model for generalization to new data.

Feature importance of: Tuned Gradient Boosting Classifier

	Imp
education_of_employee_High School	0.212819
has_job_experience_Y	0.166730
unit_of_wage_Year	0.141256
education_of_employee_Master's	0.083348
education_of_employee_Doctorate	0.070560
continent_Europe	0.063136
continent_North America	0.049956
hourly_wage	0.035837
region_of_employment_Midwest	0.035455
no_of_employees	0.023901

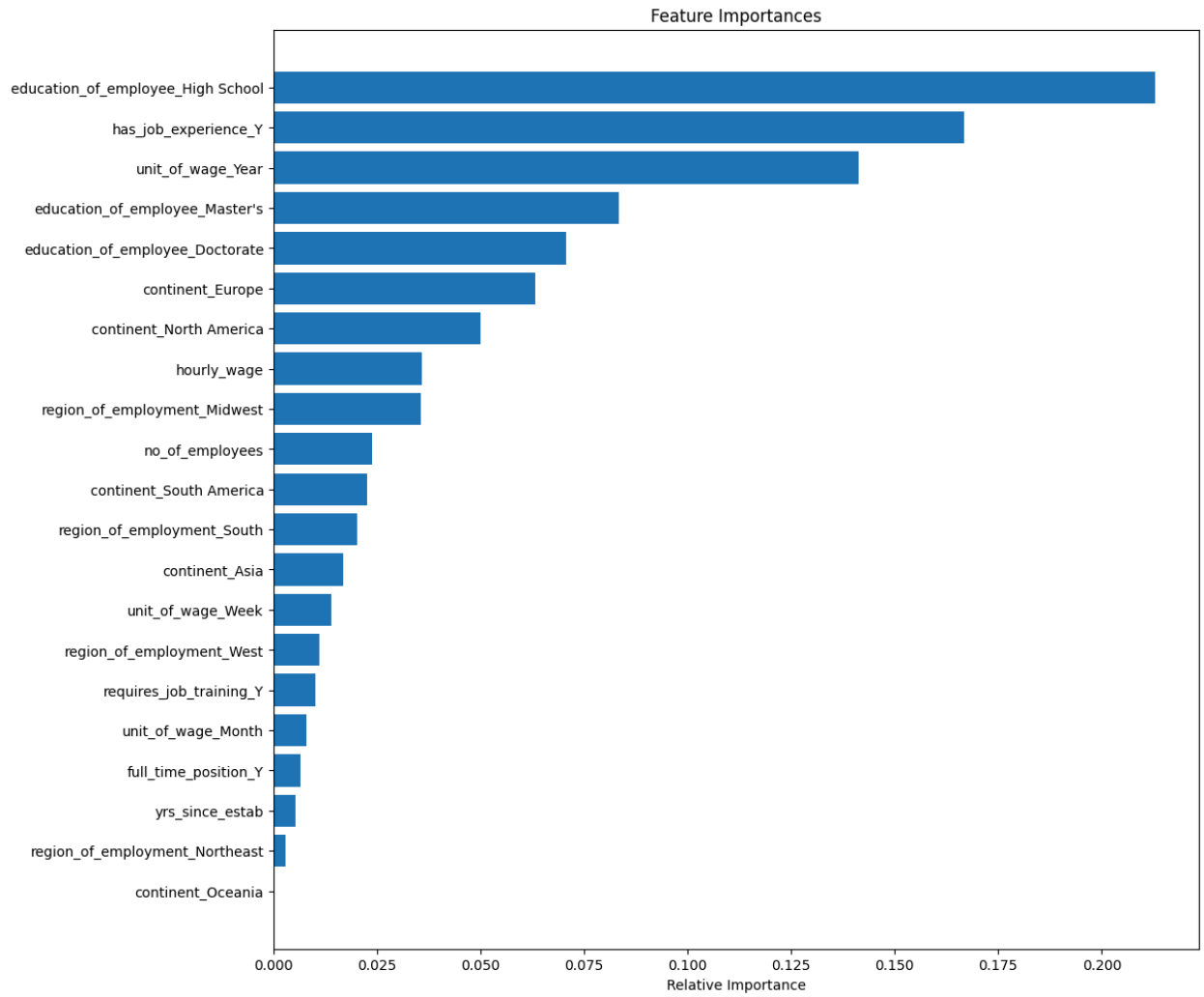


Fig 47: Feature importance of: Tuned Gradient Boosting Classifier

Observations:

The top three independent features of importance in the tuned Gradient model are education_of_employee_high_school, has_job_experience_Y, unit_of_wage_year.

Visualization of Decision Tree

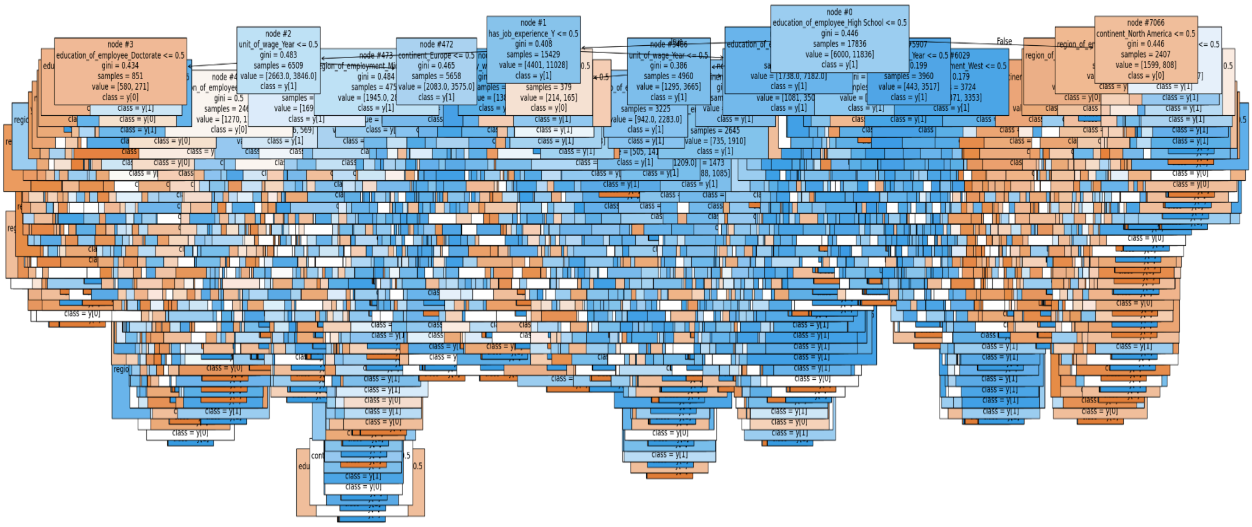


Fig 48: Visualization of Decision Tree

Feature Importance of: Decision Tree

Observations:

The top three independent features of importance in theDecision tree are hourly_wage, no_of_employees, year_since_estab.

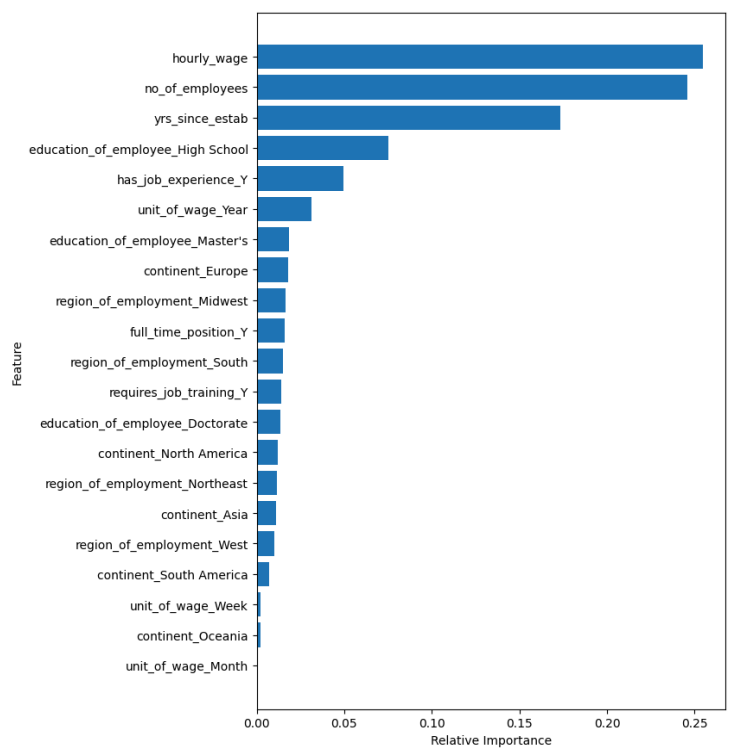


Fig 49: Feature Importance of Decision Tree

Model Building For Oversampling and Undersampling

Overview:

- Splitting the data into train, validation and test set.
  - First the data is splitted into 2 parts, temporary and test set.
  - Then we splitted temporary set into a train and validation set.
- The number of rows in train, validation and test set are
  - Number of rows in train data = 15288
  - Number of rows in validation data = 5096
  - Number of rows in test data = 5096

Model Building On Original Data

Training Performance:

```
Bagging: 0.9885406464250734
Random forest: 1.0
GBM: 0.8774730656219393
Adaboost: 0.8877571008814887
dtree: 1.0
```

Validation Performance:

```
Bagging: 0.786427732079906
Random forest: 0.8381316098707403
GBM: 0.8692714453584018
Adaboost: 0.8816098707403055
dtree: 0.7420681551116334
```

Training and Validation Performance Difference:

```
Bagging: Training Score: 0.9885, Validation Score: 0.7864, Difference: 0.2021
Random forest: Training Score: 1.0000, Validation Score: 0.8381, Difference:
0.1619
GBM: Training Score: 0.8775, Validation Score: 0.8693, Difference: 0.0082
Adaboost: Training Score: 0.8878, Validation Score: 0.8816, Difference:
0.0061
dtree: Training Score: 1.0000, Validation Score: 0.7421, Difference: 0.2579
```

Observation:

Based on the validation performance scores, the AdaBoost model has the highest recall score of approximately 0.8816, followed closely by the GBM model with a recall score of approximately 0.8693.

Model Building - Oversampled Data

Model Performance After Oversampling :

Training Performance:

```
Bagging: 0.9828599412340843
Random forest: 0.9999020568070519
GBM: 0.8489715964740451
Adaboost: 0.8501469147894222
dtree: 1.0
```

Validation Performance:

```
Bagging: 0.745005875440658
Random forest: 0.8055229142185664
GBM: 0.8407755581668626
Adaboost: 0.8437132784958872
dtree: 0.7024089306698003
```

Training and Validation Performance Difference:

```
Bagging: Training Score: 0.9829, Validation Score: 0.7450, Difference: 0.2379
Random forest: Training Score: 0.9999, Validation Score: 0.8055, Difference:
0.1944
GBM: Training Score: 0.8490, Validation Score: 0.8408, Difference: 0.0082
Adaboost: Training Score: 0.8501, Validation Score: 0.8437, Difference:
0.0064
dtree: Training Score: 1.0000, Validation Score: 0.7024, Difference: 0.2976
```

Observations:

Based on the validation performance scores after oversampling, the AdaBoost model still has the highest recall score of approximately 0.85, followed closely by the GBM model with a recall score of approximately 0.8408.

Model Building - Undersampled Data

Model Performance After Undersampling :

Training Performance:

```
Bagging: 0.9696730996455297
Random forest: 1.0
GBM: 0.7491138243402915
Adaboost: 0.7142575817250886
dtree: 1.0
```

Validation Performance:

```
Bagging: 0.6025264394829613
Random forest: 0.6745005875440658
GBM: 0.731198589894242
Adaboost: 0.7079905992949471
dtree: 0.627497062279671
```

Training and Validation Performance Difference:

```
Bagging: Training Score: 0.9697, Validation Score: 0.6025, Difference: 0.3671
Random forest: Training Score: 1.0000, Validation Score: 0.6745, Difference:
0.3255
GBM: Training Score: 0.7491, Validation Score: 0.7312, Difference: 0.0179
Adaboost: Training Score: 0.7143, Validation Score: 0.7080, Difference:
0.0063
dtree: Training Score: 1.0000, Validation Score: 0.6275, Difference: 0.3725
```

Observations:

Based on the validation performance scores after undersampling, the GBM model has the highest recall score of approximately 0.7312, followed by the AdaBoost model with a recall score of approximately 0.7080.

Overall Observations:

- It was observed that both the GBM and Adaboost models, trained on an undersampled dataset, as well as the GBM model and Adaboost model trained on an oversampled dataset, exhibited strong performance on both the training and validation datasets.
- Sometimes models might overfit after undersampling and oversampling, so it's better to tune the models to get a generalized performance
- We will tune these models using the same data (undersampled or oversampled) as we trained them on before.

Hyperparameter Tuning

Tuning AdaBoostClassifier model with Undersampled data

Best parameters are {'n_estimators': np.int64(10), 'learning_rate': 0.1, 'estimator': DecisionTreeClassifier(max_depth=1, random_state=1)} with CV score=0.9312726426438074:
CPU times: user 1.06 s, sys: 112 ms, total: 1.17 s
Wall time: 54.1 s

```
AdaBoostClassifier
estimator: DecisionTreeClassifier
DecisionTreeClassifier(max_depth=2, random_state=1)
DecisionTreeClassifier
```

The Tuning AdaBoost Classifier Performance of a Training And Validation Set

Train Performance			
Accuracy	Recall	Precision	F1_Score
0.686983	0.655179	0.699685	0.676701
Test Performance			
Accuracy	Recall	Precision	F1_Score
0.684655	0.660106	0.833148	0.736601

Table 15: The Tuned AdaBoost Classifier Performance of a Training and Validation Set(Undersample Data)

Tuning AdaBoostClassifier model with Oversampled data

Best parameters are {'n_estimators': np.int64(10), 'learning_rate': 0.1, 'estimator': DecisionTreeClassifier(max_depth=1, random_state=1)} with CV score=0.9313418217433889:
CPU times: user 1.23 s, sys: 187 ms, total: 1.42 s
Wall time: 1min 31s

```
AdaBoostClassifier
estimator: DecisionTreeClassifier
DecisionTreeClassifier(max_depth=2, random_state=1)
DecisionTreeClassifier
```

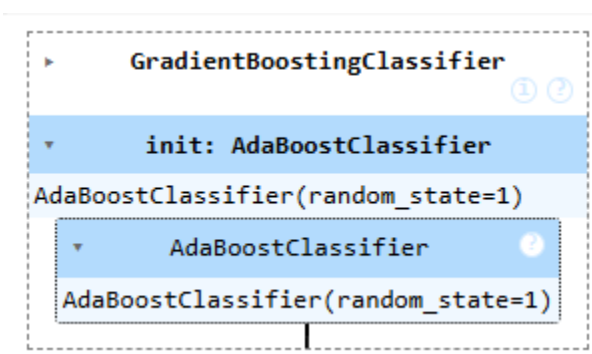
The Tuning AdaBoost Classifier Performance of a Training And Validation Set

Train Performance			
Accuracy	Recall	Precision	F1_Score
0.694613	0.904114	0.637148	0.74751
Test Performance			
Accuracy	Recall	Precision	F1_Score
0.729788	0.905112	0.745103	0.81735

Table 16: The Tuned AdaBoost Classifier Performance of a Training and Validation Set(OverSample Data)

Tuning Gradient Boosting model with Undersampled Data

Best parameters are {'subsample': 0.9, 'n_estimators': np.int64(150), 'max_features': 0.5, 'learning_rate': 0.01, 'init': AdaBoostClassifier(random_state=1)} with CV score=0.7426182072068578:
CPU times: user 4.26 s, sys: 380 ms, total: 4.64 s
Wall time: 4min 26s



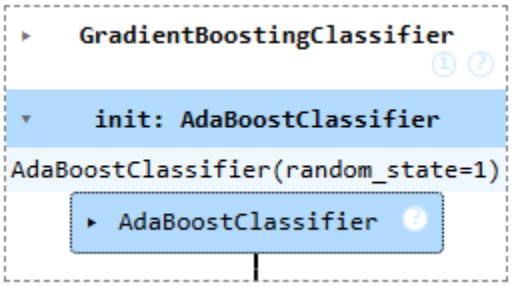
The Tuning Gradient Boost Classifier Performance of a Training And Validation Set

Train Performance			
Accuracy	Recall	Precision	F1_Score
0.705199	0.743009	0.690773	0.715939
Test Performance			
Accuracy	Recall	Precision	F1_Score
0.71978	0.73913	0.823298	0.778947

Table 17: The Tuned Gradient Boost Classifier Performance of a Training and Validation Set(UnderSample Data)

Tuning Gradient Boosting model with Oversampled data

Best parameters are {'subsample': 0.5, 'n_estimators': np.int64(100), 'max_features': 0.7, 'learning_rate': 0.01, 'init': AdaBoostClassifier(random_state=1)} with CV score=0.8777668952007834:
CPU times: user 5.26 s, sys: 499 ms, total: 5.76 s
Wall time: 5min 48s



The Tuning Gradient Boost Classifier Performance of a Training And Validation Set

Train Performance			
Accuracy	Recall	Precision	F1_Score
0.68668	0.743879	0.667516	0.703632
Test Performance			
Accuracy	Recall	Precision	F1_Score
0.71978	0.73913	0.823298	0.778947

Table 18: The Tuned Gradient Boost Classifier Performance of a Training and Validation Set(OverSample Data)

Model Comparison and Final Model Selection

Training performance comparison:

	Gradient boosting trained with Undersampled data	Gradient boosting trained with Oversampled data	AdaBoost trained with Undersampled data	AdaBoost trained with Oversampled data
Accuracy	0.705199	0.686680	0.686983	0.694613
Recall	0.743009	0.743879	0.655179	0.904114
Precision	0.690773	0.667516	0.699685	0.637148
F1	0.715939	0.703632	0.676701	0.747510

Validation performance comparison:

	Gradient boosting trained with Undersampled data	Gradient boosting trained with Oversampled data	AdaBoost trained with Undersampled data	AdaBoost trained with Oversampled data
Accuracy	0.719780	0.719780	0.684655	0.729788
Recall	0.739130	0.739130	0.660106	0.905112
Precision	0.823298	0.823298	0.833148	0.745103
F1	0.778947	0.778947	0.736601	0.817350

Fig 50: Model Comparison of Oversampled and Undersampled data

Observations:

- The AdaBoost model trained with Oversampled data has the highest Recall score of approximately 0.905 and highest f1 score approximately 0.817.

The Tuning AdaBoost Classifier Performance of an OverSampled Testing Data

Train Performance			
Accuracy	Recall	Precision	F1_Score
0.71821	0.900118	0.736538	0.810153

Table 19: The Tuned AdaBoost Classifier Performance of an oversampled Test Data

Feature Importance of OverSampled AdaBoost Classifier

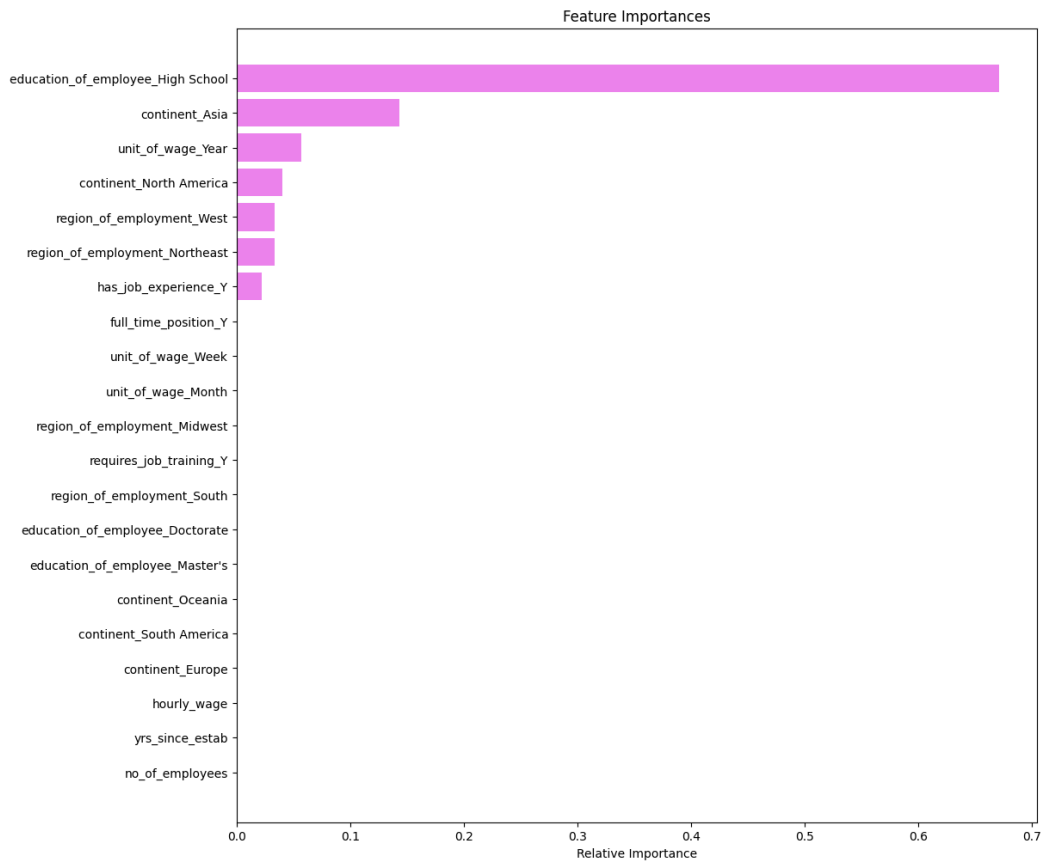


Fig 51: Feature Importance of OverSampled AdaBoost Classifier

Observations:

We can see that education_of_employee_High School_West, continent_Asia,Unit_of_wage_Year are the most important features for making predictions

Actionable Insights and Business Recommendations

Insights:

- Applicants with Master's and Doctorate degrees are far more likely to get certified compared to those with only high school education.
- Candidates with prior job experience enjoy significantly higher approval rates (~74% vs ~56% for those without).
- Applications offering yearly wages see higher approval chances compared to hourly or weekly wage offers.
- Although Asia has the largest number of applicants, approval rates are higher for applicants from Europe and Africa. Employment in the Midwest and South regions of the U.S. also corresponds with higher approvals.
- Factors like number of employees or establishment year have minimal impact compared to education, experience, and wage type.
- Base Decision Tree models are unreliable – They overfit the training data and fail to generalize on test data.
- Boosting models (AdaBoost, Gradient Boosting, XGBoost) consistently outperform bagging in terms of recall and F1-score.
- Tuned Gradient Boosting Classifier achieved the best balanced performance on the test set (F1 ≈ 0.826, recall ≈ 0.874), making it the most reliable overall model.
- Tuned AdaBoost with oversampling achieved the highest recall (≈0.905), which is very important for minimizing false negatives (not rejecting strong candidates).

Recommendation:

- Adopt the Tuned Gradient Boosting Classifier as the primary model, since it provides the best overall balance of recall, precision, and F1-score.
- Use the Tuned AdaBoost model with oversampling in cases where maximizing recall is more critical (ensuring strong applicants are not wrongly rejected).
- Bagging and Random Forest may be used as benchmarking tools but should not be the primary models due to lower recall.
- Apply oversampling techniques (like SMOTE) consistently to handle class imbalance and improve minority (denied) case prediction.
- Hyperparameter tuning should always be performed on ensemble models to reduce overfitting and stabilize performance.
- Develop feature importance, so applicants and advisors understand which factors (education, experience, wages, region) most affect approval chances.
- In the long term, explore stacked ensembles (Gradient Boosting + AdaBoost) for potentially stronger predictive power and robustness.